



DEGREE PROJECT IN MACHINE LEARNING,
SECOND CYCLE, 30 CREDITS
STOCKHOLM, SWEDEN 2022

Classification of Radar Emitters Based on Pulse Repetition Interval using Machine Learning

KTH Thesis Report

André Svensson

Author

André Svensson <andres2@kth.se>
Electrical Engineering and Computer Science
KTH Royal Institute of Technology

Host Company

Saab AB
Stockholm, Sweden

Examiner

Danica Kragić Jensfelt
School of Electrical Engineering and Computer Science
KTH Royal Institute of Technology

Supervisors

Lucas Strömberg (Saab AB)
Arvind Kumar (KTH Royal Institute of Technology)

Abstract

In electronic warfare, one of the key technologies is radar. Radar is used to detect and identify unknown aerial, nautical or land-based objects. An attribute of a pulsed radar signal is the Pulse Repetition Interval (PRI) which is the time interval between pulses in a pulse train. In a passive radar receiver system, the PRI can be used to recognize the emitter system. Correct classification of emitter systems is a crucial part of Electronic Support Measures (ESM) and Radar Warning Receivers (RWR) in order to deploy appropriate measures depending on the emitter system. Inaccurate predictions of emitter systems can have lethal consequences and variables such as time and confidence in the predictions are essential for an effective predictive method. Due to the classified nature of military systems and techniques, there are no industry standard systems or techniques that perform quick and accurate classifications of emitter systems based on PRI. Therefore, methods that allow for fast and accurate predictions based on PRI is highly desirable and worthy of research. This thesis explores and compares the capabilities of two machine learning methods for the task of classifying emitters based on received PRI. The first method is an attention based model which performs well throughout all levels of realistic noise and is quick to learn and even quicker to give accurate predictions. The second method is a K-Nearest Neighbor (KNN) implementation that, while performing well for noise-free PRI, finds its performance degrading as the amount of noise increases. An additional outcome of this thesis is the development of a system to generate samples in an automated fashion. The attention based model performs well, achieving a macro average F1-score of 63% in the 59-class recognition task whereas the performance of the KNN is lower, achieving a macro average F1-score of 43%. Future research could be conducted with the purpose of designing a better attention based model for producing higher and more confident predictions and designing algorithms to reduce the time complexity of the KNN implementation.

Keywords

Radar Warning Systems, Pulse Repetition Interval, Artificial Neural Networks, Attention, Transformers, K-Nearest Neighbour, Classification, Data Generation, Radar, Temporal Sequences,

Sammanfattning

En av de viktigaste teknikerna inom telektrig är radarn. Radar används för att upptäcka och identifiera okända, luftburna, sjögående eller landbaserade förmål. En komponent av radar är Pulsrepetitionsintervall (Pulse Repetition Intervall, PRI) som beskrivs som tidsintervallet mellan två inkommande pulser. I ett radarvarnar system (Radar Warning Receiver, RWR) kan PRI användas för att identifiera radarsystem. Korrekt identifiering av radarsystem är en viktig uppgift för elektroniska understödsmedel (Electronic Support Measures, ESM) med syfte att tillsätta lämpliga medel beroende på radarsystemet i fråga. Icke tillförlitlig identifiering av radarsystem kan ha dödliga konsekvenser och variabler som tid och säkerhet i identifieringen är avgörande för ett effektivt system. Då dokumentation och specifikationer för militära system i regel är hemligstämplade är det svårt att utröna någon typ av industristandard för att utföra snabb och säker klassificering av radarsystem baserat på PRI. Därför är det av stort intresse detta område och möjligheterna för sådana lösningar utforskas. Detta examensarbete utforskar och jämför förmågorna hos två maskininlärningsmetoder i avseende att korrekt identifiera radarsändare baserat på genererat PRI. Den första metoden är ett djupt neuralt nätverk som använder sig av tekniken "attention". Det djupa nätverket presterar bra för alla brusnivåer och lär sig snabbt att känna igen attributen hos PRI som kännetecknar vilken radarsändare och som efter träning dessutom är snabb på att korrekt identifiera PRI. Den andra metoden är en K-Nearest Neighbor implementation som förvisso presterar bra på icke brusig data men vars förmåga försämras allt eftersom brusnivåerna ökar. Ett ytterligare resultat av arbetet är utvecklingen och implementationen av en metod för att specificera PRI och sedan generera PRI efter specifikation. Attention modellen genererar bra prediktioner för data bestående av 59 klasser, med ett F1-score snitt om 63% medan KNN-implementationen för samma uppgift har en lägre träffsäkerhet med ett F1-score snitt om 43%. Vidare forskning kan innefatta utökad utveckling av det djupa,

neurala nätverket i syfte att förbättra dess förmåga för identifiering och metoder för att minimera tidsåtgången för KNN implementationen.

Nyckelord

Aritficiella Neurala Nätverk, Tidsserie data, Puls Repetitions Intervall, Attention, Transformers, K-Nearest Neighbor, Klassificering, Datagenerering, Radar

Acknowledgements

I would like to thank my supervisor at KTH, Arvind Kumar, for his insights, recommendations and perspective on the thesis project.

I would also like to thank Lucas Strömberg, my supervisor at Saab, for his guidance and support during this thesis project.

Lastly, I would like to thank my family and especially my fiancé, Linnea Nordahl, for her invaluable support throughout the thesis project.

Acronyms

EW Electronic Warfare

ELINT Electronic Intelligence

EM Electromagnetic

RWR Radar Warning Receiver

ESM Electronic Support Measures

PRI Pulse Repetition Interval

TOA Time of Arrival

ANN Artificial Neural Network

CNN Convolutional Neural Network

MLP Multilayer Perceptron

ReLU Rectified Linear Unit

LSTM Long-Short Term Memory

KNN K-Nearest Neighbort

DTW Dynamic Time Warping

LBK Lower Bound Keogh

Contents

1	Introduction	1
1.1	Electronic warfare	1
1.2	Problem	2
1.3	Purpose	3
1.4	Research Question	3
2	Background	4
2.1	Radar technology	4
2.1.1	Pulse Repetition Interval	5
2.2	Artificial Neural Networks	6
2.2.1	Multilayer Perceptron	7
2.2.2	Activation Function	8
2.3	Transformers	9
2.3.1	Layer normalization and Feed Forward	12
2.3.2	Sequence Embedding	13
2.4	K-Nearest Neighbor	14
2.4.1	Dynamic Time Warping	15
2.5	Related works	16
3	Methods	18
3.1	Data Generation	18
3.1.1	Modulation types	19
3.1.2	Noise	21
3.1.3	Data sets	21
3.2	Attention Based Model Implementation	24
3.2.1	Transformer Encoder	25
3.2.2	Multi Layer Perceptron	27

3.3	K-Nearest Neighbor	27
3.4	Performance metrics	27
3.5	Hardware and Software	29
4	Results and Analysis	31
4.1	Attention model architecture	31
4.1.1	Embedding the inputs leads to better predictions	31
4.1.2	Increasing the number of attention heads and encoder blocks does not improve predictive qualities	32
4.2	The attention based model is a good predictor for multiple classes . . .	32
4.3	K-Nearest Neighbor is a limited predictor for multiple classes	33
5	Discussion	36
5.1	The role of the embedding block	36
5.2	The impact of attention heads and encoder blocks	36
5.3	The strength and weaknesses of the attention based model	37
5.4	The struggling K-nearest neighbor	38
5.5	Generating large amounts of data	38
5.6	Impact of the mean	39
6	Conclusions and Future Work	40
6.1	Conclusions	40
6.2	Limitations	40
6.3	Future work	41
6.4	Reflections	41
6.4.1	Sustainability, Societal and Ethical aspects of thesis	41
	References	43

Chapter 1

Introduction

This chapter will firstly present a brief history on the topic of electronic warfare, its subdomain electronic intelligence and briefly introduce the subdomain radar signal analysis. Secondly, the main problem this thesis tries to answer is presented. Lastly the purpose and research question is presented.

1.1 Electronic warfare

Electronic warfare (EW) is defined as actions taken by conflicting sides for the detection, disruption and information gathering in order to achieve control or insights of opponents weapons, vehicles and electronics while also defending your own electronic systems [29]. There are a wide array of methods to achieve control of the electromagnetic (EM) spectrum. One method is to use jamming techniques in order to disrupt enemy reconnaissance and surveillance of the EM spectrum. For example, during second world war (WWII), the British Airforce spread thin strips of tinfoil during air raids over Hamburg in order to disrupt German radar stations, which proved successful in reducing British losses and imposing control over the EM spectrum [29].

Electronic intelligence (ELINT) is an area of EW defined as the collection and analysis of electronic signals. The information gathered can be used to define and understand signal emitter operation and purpose in order to derive as to whether the emitter is part of a weapons platform or surveillance system. Part of ELINT systems are Radar Warning Receiver (RWR) systems. The purpose of the RWR is to analyze information

received in order to determine the source and threat level of the source. Radar is one of the most commonly used technologies in EW and is a key factor in information transmission and retrieval. All radar produce a radio frequency signal that is unique to itself and it is therefore possible to gather information and assert the characteristics and operation of aforementioned radar. One part of the radio frequency signals is the Pulse Repetition Interval (PRI), which is defined as the time difference between pulses in a repeating signal. The information found in the PRI can be used to predict its source and multiple methods for approaching this task have been proposed. Examples of methods for PRI analysis and classification are the use neural networks, machine learning algorithms and statistical analysis. This thesis will evaluate and compare the capabilities of an attention based neural network with the k-nearest neighbor machine learning algorithm.

1.2 Problem

Speed, flexibility and information are important aspects in in electronic warfare. The ability to accurately and in a timely manner be able to know where the enemy is and what measures they take against your forces are paramount. This is the case in all levels of military operations and holds true in the domain of EW. Much of the information gathering, weapons platform targeting and counterintelligence is conducted in the electromagnetic spectrum. The development and production of efficient, scalable and accurate methods are therefore of great interest. For instance, in the case of a fighter pilots ability to respond correctly in a given situation, information regarding enemy platform function and intent needs to be supplied quickly and accurately. However, there are at any instance a wide range of signals in the EM spectrum and the methods used need to be able to differentiate between signals, whether they be military or civilian in nature. The problem of classifying signals becomes increasingly difficult as modern radar systems can smoothly change their mode of operation and therefore avoid broadcasting the enemy information about the platform operations and intent. Improving passive radar identification systems for successful classification of new radar may be a difficult task, in particular for rule-based systems. This, in combination with the abundance of radar signal data, both real and generated, makes machine learning a natural tool for easily improved upon identification systems.

1.3 Purpose

The purpose of this thesis is to explore and evaluate the capabilities of an attention based model for the classification of emitter PRI data. The intention is to implement the model and evaluate the overall accuracy, time for training, and performance and compare it to a classical machine learning algorithm namely the k-nearest neighbor algorithm.

1.4 Research Question

The main research question that this thesis is based upon answering is how well suited a transformer based model is for the classification of emitters based on their PRI compared to the k-nearest neighbor algorithm.

Chapter 2

Background

This chapter will include a detailed description of radar technology, its use in electronic warfare and aspects that are of relevance to this thesis. The chapter also includes a historical and theoretical background to neural networks where the main focus will be the transformer neural network and its methods. The chapter concludes with a description of the K-Nearest Neighbor and related algorithms used in this thesis project.

2.1 Radar technology

Radar technology can be used in a wide array of areas; from automated vehicle guidance systems, door openers, drones and tracking. For instance, with radar it is possible to track an objects movements, distance, position, speed and angle. During the second world war (WW2) that the potential of the technology was recognized. For example, before and during WW2, the United States developed radar technology that allowed for the tracking of military ground, sea and air vessels. The development has continued at a rapid pace and radar can be found in guidance systems for spacecraft or radar guided weapons systems [32]. In its most simplistic form, a radar system can be described as a system that emits an electromagnetic wave. The wave then reflects against a surface and returns to the system where information of the reflection points position can be extracted. One of the more important aspects is that a timed delay is placed in between bursts, so that one burst can travel from the emitter, reflect of an object, and travel back to the radar before the next wave is emitted. This is done in order to avoid destructive interference of bursts [32]. The goal is not always

to emit and analyze the returning emission. The waves that are emitted by other systems can be collected and analyzed by passive systems in order to infer the type of radar system and its position. This is what allows for military systems to employ countermeasures based on information given by systems such as the Radar Warning Receiver (RWR). RWR is a system that detects radar signals and performs signal identification to determine the type of emitter the signal originates from in order to deploy the correct countermeasures [23]. A big part of a RWR system is the processing and identification of Pulse Repetition Intervals (PRI).

2.1.1 Pulse Repetition Interval

The Pulse Repetition Interval (PRI) is defined as the time between pulses, radiated from a source such as a radar system. The time interval between pulses can vary and a sequence of pulse intervals are defined as a pulse train or PRI vector. The time interval between pulses are usually in the order of microseconds and receiver systems need to be robust and sensitive enough to accurately capture the entire pulse train. The PRI vectors can be visualized and in some cases identified based on their appearance. In the visualization of PRI vectors, each PRI is represented in the order of arrival on the x-axis and the time between pulses on the y-axis, one can see that the PRI vectors form identifiable shapes representing the modulation types illustrated in figure 2.1.1 [30]. For example, a radar emitting with a PRI vector representation that is visualized as a sine shape is declared as an emitter in sine modulation mode. The modulation gives insight into the emitter characteristics and operation and allows for classification between modes of operation and emitters. The emitter can be in different modes of operation and switch between them seamlessly. The characteristics of emissions varies with the systems modes and features of interest can be, but not limited to, pulse width, frequency and bandwidth.

Noise

In a connected world where multiple technologies are emitting signals, both in the military and civilian domains, difficulties arise in the deinterleaving of radar signals. Multiple emitter signals can be interpreted as originating from the same emitter or overlap each other resulting in noisy data. There are three types of noise considered in this thesis, namely spurious pulses, pulse drop, and Gaussian noise. Spurious pulses occur if, for instance, a signal is recorded over an area with multiple sources and

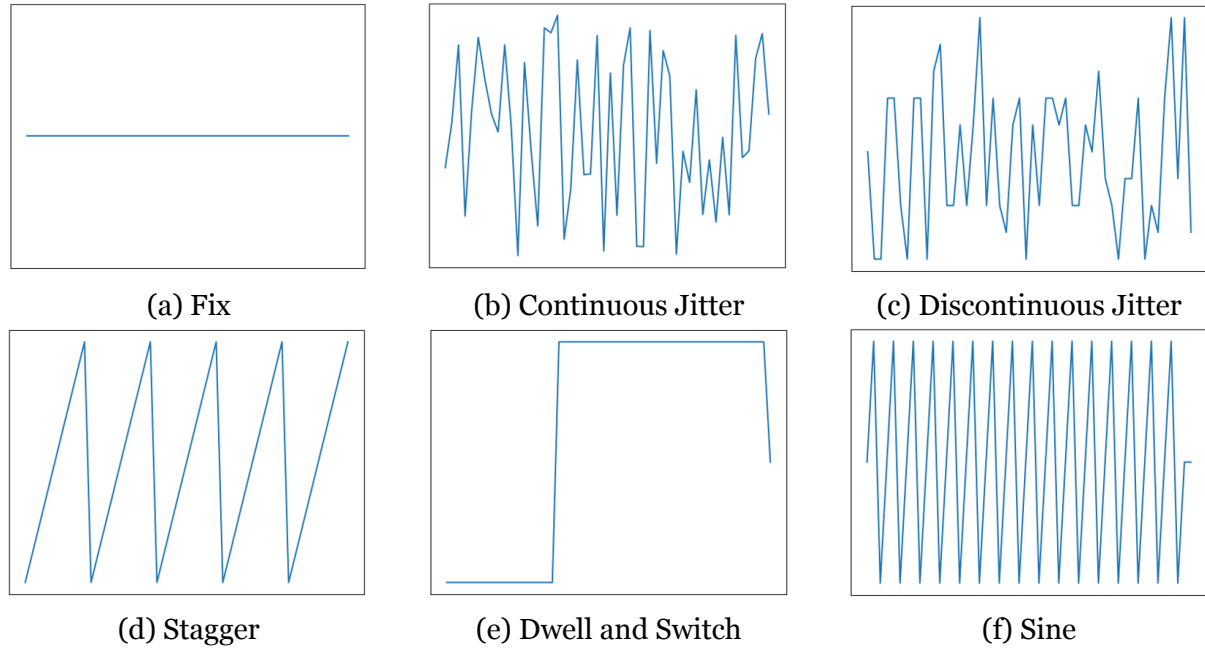


Figure 2.1.1: Examples of modulation types

the deinterleaving process is unable to accurately differentiate between sources. This might lead to the signals overlapping, causing noise in the form of spurious pulses in the pulse train [2]. Spurious pulses are not part of the true pulse train and can be defined as PRI with a time of arrival (TOA) higher than both the previous and following PRI. Pulse drops instead occur when the receiver is unable to recognize a PRI as part of the pulse train, resulting in the loss of PRI part of the true pulse train [2]. Lastly, Gaussian noise in the PRI can be caused by a number of reasons. Some sources of Gaussian noise in the collected pulse train is the cosmic background radiation, civilian cellphone use or that the hardware used for collecting the signal causes noise due to EM interference between components [32]. Gaussian noise can affect the deinterleaving of a pulse train, and the standard deviations of Gaussian noise is not uniform across modulation types and can be generated accordingly (See table 3.1.8).

2.2 Artificial Neural Networks

Artificial Neural Networks (ANN) is an area of research within the field of machine learning. ANNs were originally modeled after how biological brains work starting with the modelling of the neuron [1]. The simplest ANN is a single neuron that either gets activated due to strong enough input, or remains at rest if the input is too weak [12]. Layers of connected neurons receive and output signals to and from each

other and gradually, over time these connections are reinforced and become stronger as learning takes place. However, modern neural networks are, for the most part, more complex with multiple layers of connected neurons used for multiple, different applications.

2.2.1 Multilayer Perceptron

The Multilayer Perceptron (MLP) is a fully connected neural network consisting of an input layer, a single or multiple hidden layers, and an output layer as illustrated in figure 2.2.1. MLP's are capable of learning the mappings of non-linear functions and are mostly used for regression and classification tasks [12]. The layers consist of nodes or neurons which in turn are comprised of a bias, a connection to each following neuron and an activation function. The connections are what is colloquially called the weights and it is these weights and biases that are tuned during the network training. The process of input to output of a neuron can be summarized as thus; the sum of the weighted inputs to the neuron and the bias are passed through an activation function which are then forwarded as the input to the neurons in the following layer. The process is presented in equation 2.1 where x_j are the inputs, W_j are the weights, b_j are the biases, Φ is the activation function and y_j is the output for n number of units in the layer.

$$y_j = \Phi\left(\sum_j^n x_j w_{ij} + b_j\right) \quad (2.1)$$

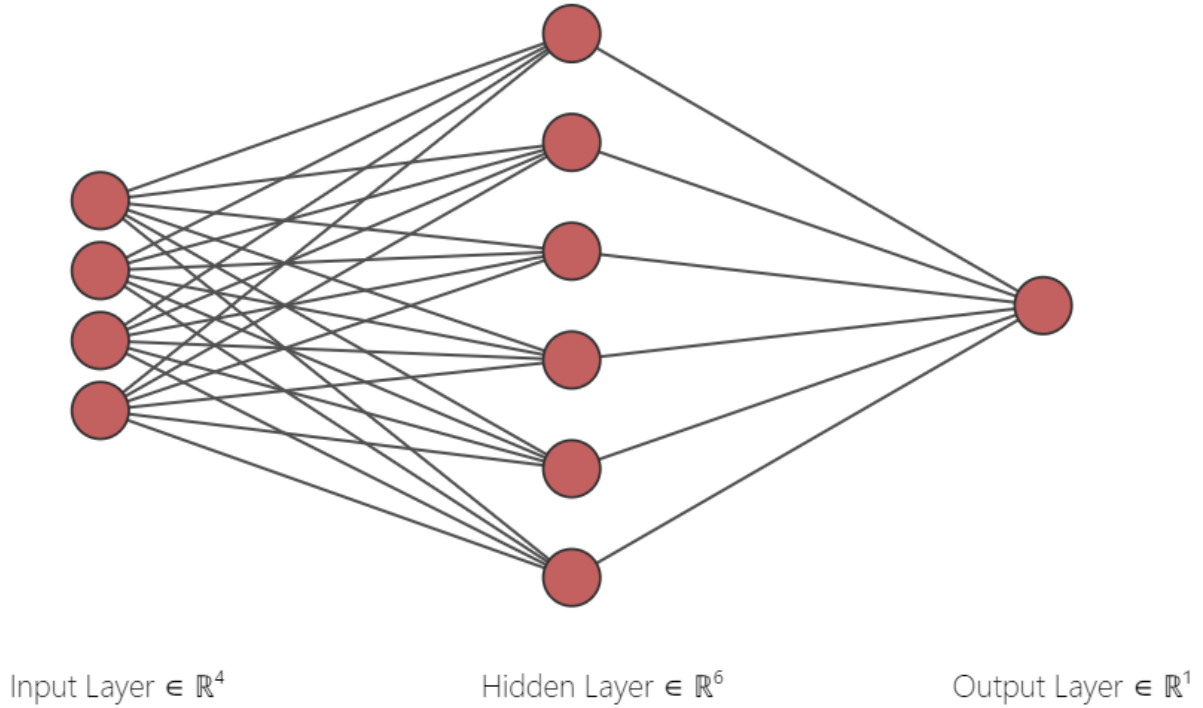


Figure 2.2.1: Schematic of a Multilayer Perceptron with one hidden layer

2.2.2 Activation Function

In order for a neural network to learn, connections between neurons need to be strengthened during training. The functions that govern the process of strengthening connections are the activation functions. The activation function is any function f that receives the sum of the neurons input and depending on that value, transforms the value before being forwarded to the following neurons it is connected to. Examples of activation functions are the sigmoid (equation 2.2), Rectified Linear Unit (ReLU) (equation 2.3), LeakyReLU (equation 2.4) and tanh (equation 2.5), functions. The behavior of the functions are illustrated in figure 2.2.2. The sigmoid and tanh activation functions have traditionally been the standard choice of activation function but recently the ReLU activation function has become the de facto choice for neural networks due to its simplicity and quick computation time [12]. However, using ReLU as an activation function can lead to gradient saturation, meaning that the gradient of the neuron converges to zero and becomes "dead". The dead neuron will stop learning and always output an activation value of 0. To counteract the problem of dead neurons, one can use the Leaky ReLU activation function due to its ability to output small

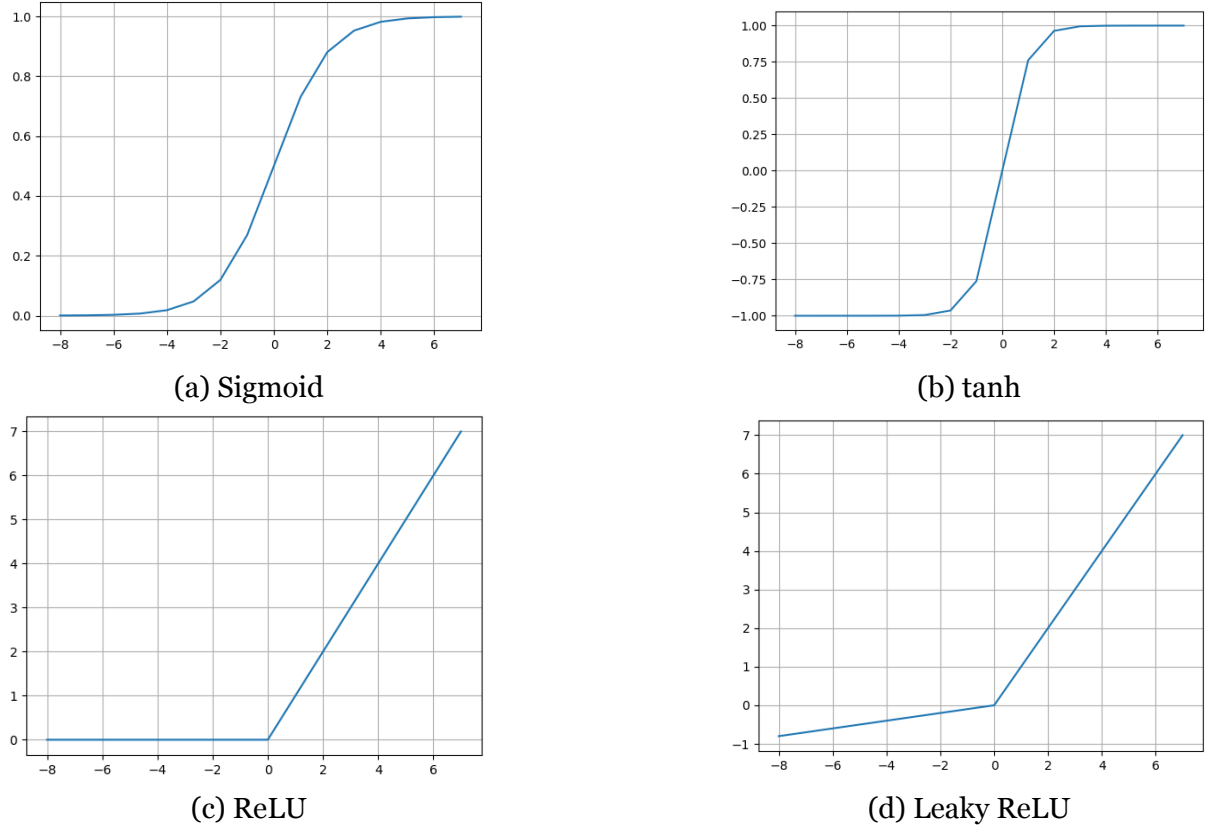


Figure 2.2.2: Graphs of activation functions

negative values for input less than 0. [22].

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.2)$$

$$ReLU(x) = \max(0, x) \quad (2.3)$$

$$LeakyReLU(x) = \begin{cases} 0.001x & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases} \quad (2.4)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (2.5)$$

2.3 Transformers

Since its conception, the transformer has proven to be a very useful type of neural network in the domain of Natural Language Processing (NLP). For instance, the BERT-

model which has a very similar architecture to the original transformer model, is a pre-trained model that can be used for a number of natural language processing tasks [9]. A pre-trained network is a network that does not need much new data to successfully become proficient in a new task domain. The success of BERT has led to a number of works based on the BERT-model. For example, it has been shown that the pre-trained BERT-model is significantly undertrained and a new method has been proposed called the "Robustly Optimized BERT Pretraining Approach", which improves performance of BERT across multiple metrics [21]. The transformer has also been a key factor in the success of other models such as the GPT-2, which is a transformer based, pre-trained model for generating, interpreting and translating text. An example of the complexity and strength of the GPT-2 is that it has been shown to be able to generate poems of such quality that make them, in some aspects, indistinguishable from poems written by humans [20]. The transformer is a neural network architecture that is based on the concept of self-attention, a method from which the network is able to learn the meaning behind sequences in the input and output [33]. One of the key advantages of transformers compared to other neural network architectures is that the problem of vanishing gradients is avoided entirely, giving the network the ability to process longer sequences of data as input. Transformers are also flexible enough to be applied to transfer learning tasks, where the network can be repurposed with a small amount of new training data [37]. The transformer architecture is composed of an encoder block and a decoder block, each with their own multi-head attention blocks, feed forward blocks, embedding blocks with positional encoders and layer normalization blocks with residual connections. The architecture is illustrated in figure 2.3.1. The following subsections explain the operation of the blocks.

Multi-Head Attention

The attention mechanism is described as the calculation of the weighted sum of keys, values and queries. The value of the weights in the network is calculated using a compatibility function for each query to each corresponding key. The attention mechanism is also called "Scaled Dot-Product Attention" where the dot product of all queries and keys is divided by the square root of the dimensions of the keys on which softmax is then applied in order to get the proper weight values. Equation 2.6 represents the attention calculation where Q is the query, K is the key, and V is the value vectors of the input respectively [34]. The attention mechanism is further

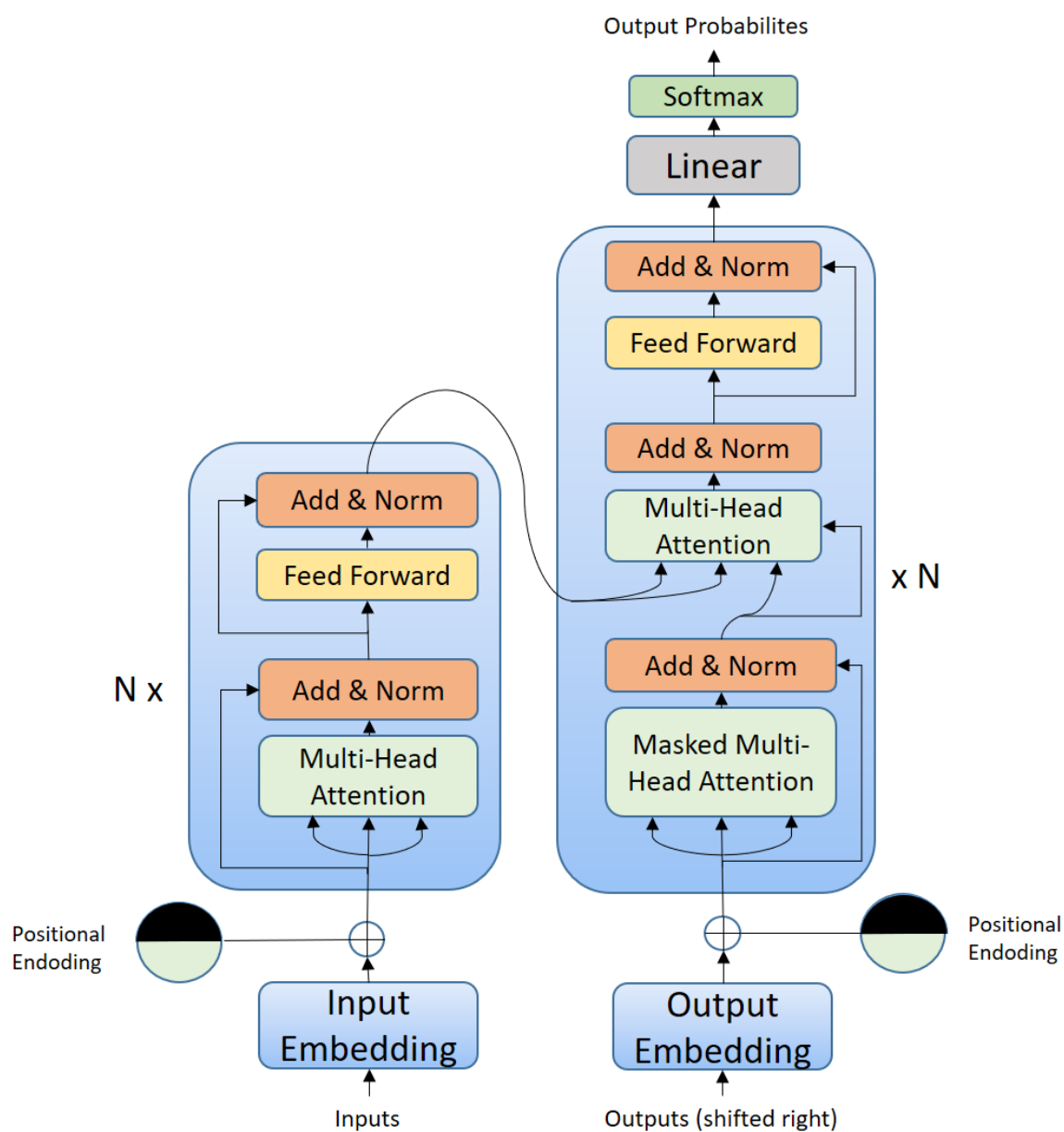


Figure 2.3.1: The transformer architecture in the original implementation [34]

improved when one introduces multiple linear projections of the keys and queries, use the projections and perform the attention function in parallel. The output of this calculation is then linearly projected a second time. This is what allows for the model to garner information from different subspace representations at varying positions from the same input [34]. The number of heads can vary and it is not yet completely understood how number of attention heads affect performance [8]. Research has shown that some attention heads are more important than others in translation tasks and that a majority can be pruned with a relatively small loss in performance [36]. The heads have been described as having one or more unique specializations such as attention to neighbouring words and to tokens in syntactic relations [36]. The equations 2.7 and 2.8 show how the output of the Multi-Head Attention mechanism is calculated.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (2.6)$$

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O \quad (2.7)$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (2.8)$$

Masking

An important part of the decoder block in the transformer architecture is the masking of sequences. The decoder inputs at a position i are masked, only allowing the attention heads to attend to positions preceding position i . This is what makes the decoder block learn to predict the next value in a sequence [34].

2.3.1 Layer normalization and Feed Forward

[4] Layer normalization is an alternative to the more common batch normalization layer that is comparatively quicker and in the case of training and testing Recurrent Neural Networks, improves performance [4]. Layer normalization can be summarized as calculating the mean and variance of the summed inputs for the hidden layer. The formula for calculating the mean and variance are presented in equations 2.9 and 2.10 where H is the number of hidden units in each layer l , μ is the mean, σ is the variance and a_i is the input to unit i in the layer [4].

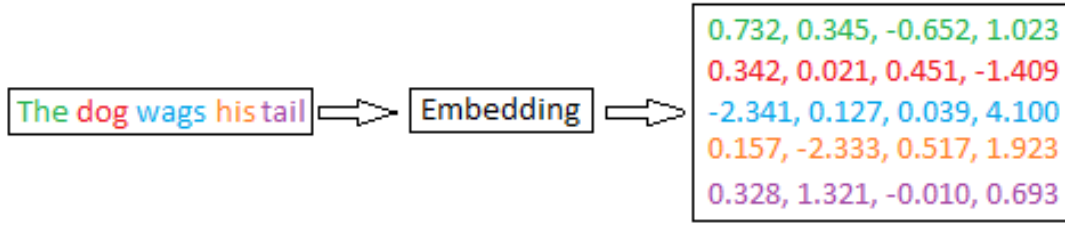


Figure 2.3.2: Illustration of how a word sequence is transformed during embedding

$$\mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l \quad (2.9)$$

$$\sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu_i^l)^2} \quad (2.10)$$

The feed forward block is essentially two fully connected linear layers with a ReLU activation function between them. Equation 2.11 presents how the output is computed in the feed forward block [33].

$$\text{FeedForward}(x) = \text{ReLU}(W_1 x + b_1) W_2 + b_2 \quad (2.11)$$

2.3.2 Sequence Embedding

This subsection will present the method of embedding sequences as was presented in the original transformer architecture and an alternative method, Time2Vec which is designed for encoding temporal sequences with positional information.

Transformer embedding The pre-processing of the data in the original transformer implementation for NLP tasks required transforming the sequence values into the embedding space representation. Every word in the sequence is transformed and then positional information is injected into the embedded transformation using either the sine or cosine function. An example of how a word-sequence is transformed in the embedding step is presented in figure 2.3.2. The positional encoding functions are defined in equations 2.12, 2.13.

$$PE_{(pos,2i)} = \sin(pos/10000)^{2i/d_{model}} \quad (2.12)$$

$$PE_{(pos,2i+1)} = \cos(pos/10000)^{2i/d_{model}} \quad (2.13)$$

Time2Vec One method for pre-processing time series data is the Time2Vec method. It is described as providing a model-agnostic vector representation for time which lends itself to easily be implemented in combination with varying model architectures [17]. The method has been shown to improve the results of models in multiple tasks for event predictions in combination with an LSTM based architecture. The Time2Vec function can be seen in equation 2.14 where $\mathbf{t2v}(\tau)[i]$ is the i^{th} element of $\mathbf{t2v}(\tau)$, F is the activation function, and ω_i and φ_i are learnable parameters. Originally, the F activation is defined as the sine function but it can be defined as the cosine or other activation functions and produce similar results.

$$\mathbf{t2v}(\tau)[i] = \begin{cases} \omega_i\tau + \varphi_i, & \text{if } i = 0 \\ F(\omega_i\tau + \varphi_i), & \text{if } 1 \leq i \leq k. \end{cases} \quad (2.14)$$

2.4 K-Nearest Neighbor

The K-Nearest Neighbors (KNN) algorithm is a statistics based supervised learning method used for regression and classification tasks. The KNN algorithm can be described as an algorithm that, for any given input, iterates over a set of data in order to find the k-nearest samples of the set that are closest in distance to the input. An example of a binary class problem that can be solved using KNN is illustrated in figure 2.4.1. Once the neighbors are found, the output is a value derived from the majority vote of the labels closest to the input. The value K determines the number of neighbors needed to be found before the majority vote is made [7]. The KNN algorithm is described as a "lazy learner", meaning that the training set is only stored and used during training. Therefore, memory limitations plays a large part in tasks where the algorithm might be used. It is used for recommendation systems, pattern recognition, prediction tasks and more [38].

There are multiple methods for calculating the distance or similarity between samples. The classical approach to measuring the similarity of two sequences a and b is

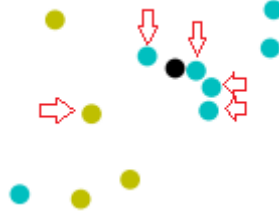


Figure 2.4.1: Image of binary classification problem where the KNN has identified the $K = 5$ closest neighbors, marked by the red arrows

calculated using the the euclidean distance. The euclidean distance calculation is presented in equation 2.15. However, other methods for measuring similarity have proven to be successful and in some cases outperforming the euclidean distance measurement. One of them is the Dynamic Time Warping (DTW) algorithm [28].

$$f(a, b) = \sum_{i=1}^n (x_i(a) - x_i(b))^2 \quad (2.15)$$

2.4.1 Dynamic Time Warping

Dynamic time warping (DTW) is a similarity measure primarily used for temporal sequence data in the domain of machine learning. Originally developed for use within the task domain of speech recognition, it has found much success in other fields such as data mining and signal analysis [6]. DTW takes two temporal sequences and calculates a distance matrix comprised of the distances between individual points in the sequences. Afterwards, the algorithm performs a search through the matrix where the smallest values are added to the path until the algorithm has traversed the entire matrix, as can be seen in table 2.4.1. The algorithm is presented in equation 2.16 where a_i and b_j are elements in sequences A and B of length i and j respectively. D is the computed or partially computed distance matrix. The matrix calculation starts from index $(1, 1)$ and ends at index (i, j) . DTW has been shown to be a more robust metric for measuring similarity than euclidean distance since it is able to find similarity between temporal sequences that are phase shifted in time and therefore produces predictions with lower error rates [18]. A problem with a DTW is that the time complexity of the algorithm is quadratic, $\mathcal{O}(MN)$ for sequences of length M and N , meaning that computation times increase with increasing sequence length. There are numerous algorithms for reducing computation times. One of which is the Lower Bound Keogh method [18].

2	9	9	7	6	10
3	8	6	6	7	10
5	8	4	8	11	9
7	6	4	10	11	8
1	2	6	6	7	12
	3	5	1	2	6

Table 2.4.1: The distance matrix of sequence A, y-axis, and sequence B, x-axis. The sequence of grey cells starting from the upper right corner mark which values represent the best alignment for the two sequences A and B.

$$D(i, j) = |a_i - b_j| + \min \begin{cases} D(i-1, j) \\ D(i-1, j-1) \\ D(i, j-1) \end{cases} \quad (2.16)$$

Lower Bound Keogh

Lower Bound Keogh (LBK) is an algorithm used to speed up the process of finding nearest neighbors in combination with DTW. LBK filters the poorly matching temporal sequences so that only the ones that are possible matches are used in the DTW algorithm. Essentially, the algorithm uses the input sequence Q and from that creates two boundary sequences L and U. The distance between the candidate sequence C to the L and U sequences is calculated and used to decide whether the candidate sequence is of interest. Therefore, LBK allows for a faster KNN using DTW by filtering out candidates that are not a potential match to the input. The calculations of the LBK method is shown in equation 2.17

$$LBK(Q, C) = \sum_{i=1}^n \begin{cases} (q_i - U_i)^2 & \text{if } q_i > 0 \\ (q_i - L_i)^2 & \text{if } q_i < L_i \\ 0 & \text{otherwise} \end{cases} \quad (2.17)$$

2.5 Related works

Much research has been conducted within the area of PRI-analysis, both in the domains of statistical modeling and deep neural networks. One of these methods is a deep learning-based multi-task learning method for the deinterleaving and PRI

modulation recognition of radar pulses [14]. The proposed method uses the complex wavelength transform (CWT) in combination with a convolutional neural network to classify the images produced by the CWT. The proposed method shows great results in the purposes of deinterleaving PRI but for the task of PRI modulation recognition the performance drops, and even more so for increasing levels of noise [14]. One of the earlier methods for automatic PRI modulation type recognition is the use of a 3-layer MLP which was shown to produce accurate modulation type classifications in the range of 98% to 100% [24]. The result from the 3-layer MLP is from a modulation type based quaternary classification task. Another interesting method for PRI modulation type recognition is the decimated Walsh-Hadamard transform (DWHT) [10]. The Walsh-Hadamard transform can be described as a class of Fourier transforms. In DWHT, the generated WHT representations are decimated by a rate of 4, this means that only every fourth sample in the representation is kept and the rest removed. The decimation alleviates the problem of spurious pulses and parts of the signal in order to get a less noisy input [10]. The DHWT method achieve a top 85% accuracy for PRI modulation type recognition. Vector neural networks (VNN) have also shown great results when applied to the task domain of emitter type classification using a combination of radio frequency (RF), pulse width (PW) and PRI data [31]. The VNN uses new vector-type backpropagation (NVTBP) as it's learning algorithm to make use of interval-value input data as well as scalar input data in order to make use of more data and achieve higher classification accuracy [13]. The VNN implementation achieves a top accuracy of 99% for data without noise in the two class recognition task [31]. Previous works at Saab has shown great potential for deep learning models in the case of PRI modulation type recognition with a Long-short Term Memory model (LSTM) achieving a top accuracy of 99% and a CNN implementation achieving a top accuracy of 86% [25], [5].

Chapter 3

Methods

This chapter describes the methods employed in this project for data generation, model implementation and model evaluation. Firstly, the generation of data is presented with information regarding number of samples, noise levels and other parameters. In the next section the methods and implementation of the attention based model is presented with descriptions and motivation of design choices. Thirdly, the KNN using DTW and LBK implementation is presented. Lastly, the methodology of testing and evaluating the two methods and components is presented.

3.1 Data Generation

The emitter data is defined as per information given in documents supplied by Saab and Försvarets radioanstalt (FRA). Scripts for generating the different modulation types are also supplied with methods for injecting noise to the samples. Six different modulation types are used for the emitter definitions; fix, continuous jitter, discrete jitter, sine, dwell and switch, and stagger. The length of the emitter vectors are randomized with a length between the minimum and maximum PRI length to mimic the limited amount of pulse-train data that are collected in a given real world situation. The starting index of the true PRI in a sequence is also randomized with the condition that the random index should allow for the minimum length of PRI to be generated. For example, a vector of maximum of 10 and minimum PRI length of 3 will not allow for the true PRI sequence to start at indices 9 and 10 since the final vector would be longer than the maximum allowed length. The data is generated in order of noise levels, modulation types and number of samples where these are defined in libraries

based on the supplied documents and recommendations from experts at Saab. The attention based model expects inputs to be of uniform size, the generated vectors are therefore padded by appending the vector mean after noise injection until the vector is the expected size.

3.1.1 Modulation types

The modulation types have different characteristics based on their modes of operation and have been defined accordingly. For instance, in real world scenarios, pulse trains of the dwell and switch modulation type have a small amount of gaussian noise and this is modeled during generation. Following is a description of the modulation types and their parameters that are used during generation.

Fix The fix modulation type is the most simple and can be described as PRI where each following PRI holds the same value as the previous PRI. It is modeled as a function $f(x) = x$ for any real number $x > 0$, and an example of parameters used to generate the fix modulation type can be found in table 3.1.1.

minLen	5
maxLen	50
maxPri	801.0 μs

Table 3.1.1: Parameters for generating the fix modulation type

Continuous Jitter The continuous jitter modulation type is stochastic in nature where pseudo-random values are chosen from within the range of an interval. The interval is defined by the mean and standard deviation, and an example of parameters used to generate the continuous jitter modulation type can be found in table 3.1.2.

minLen	5
maxLen	50
minPri	713.12 μs
maxPri	889.97 μs

Table 3.1.2: Parameters for generating the continuous modulation type

Discrete Jitter Discrete jitter is defined much like the continuous variant but PRI values are defined by a pre-determined schema. The interval and schema is defined by the mean and standard deviation. An example of parameters used to generate the discrete jitter modulation type can be found in table 3.1.3.

minLen	5
maxLen	50
minPri	715.0 μs
maxPri	881.2 μs

Table 3.1.3: Parameters for generating the discrete jitter modulation type

Stagger The stagger modulation type is defined by a pre-determined pattern. In the generation it is defined as $f(x) = x_1, x_2, x_3, \dots, x_n$ where the pattern repeats. An example of parameters used to generate the staggered modulation type can be found in table 3.1.4.

minLen	5
maxLen	50
cycle	1180.0 1209.0 1240.0 1209.0 μs

Table 3.1.4: Parameters for generating the stagger modulation type

Sine The sine modulation type is defined as values that, when one plots their values against their index of occurrence, makes up a graph that looks like a sine curve. The function used to generate PRI of this modulation type is $F(n) = PRI_0 + A \sin(\omega x_n + \phi)$. In the generation it is defined by a maximum and minimum value for the curve and the number of values in each period. An example of parameters used to generate the sine modulation type can be found in table 3.1.5.

minLen	5
maxLen	50
minPri	805.3 μs
maxPri	819.7 μs
freq	50Hz

Table 3.1.5: Parameters for generating the sine modulation type

Dwell and switch The dwell and switch modulation type is comprised of sequences of PRI with the same value for a pre-defined number of PRI. In the generation it is defined as values to switch between PRI of the number of PRI per value. An example of parameters used to generate the dwell and switch modulation type can be found in table 3.1.6.

minLen	5
maxLen	50
PRI values	[1300, 1350, 1215] μs
Pulses per PRI	[5, 6, 8]

Table 3.1.6: Parameters for generating the dwell and switch modulation type

3.1.2 Noise

The different noise levels for data generation are set according to information supplied by experts from Saab in order to as accurately as possible model the noise present in real world data. Examples of noise levels are presented in tables 3.1.7 and 3.1.8.

Pulse drop Dropped pulse noise is injected to each PRI vector by the probability that a PRI is lost in the pulse train. The probabilities that a pulse is dropped is normally in the range 0% $\rightarrow$ 30% but could be as much as up to 50%.

Spurious pulses The schema for the injection of spurious pulses to the pulse train is defined as the percentage of the length of each vector that should be spurious. Normally spurious pulse injection is modeled as a Poisson process but due to the large amount of vectors to be generated the injection of spurious pulses is modeled as deterministic process.

Gaussian Noise The gaussian noise injection is defined by the standard deviation where a vector of the same length as the input is generated randomly and then added on top of the input.

3.1.3 Data sets

Two data sets are used in the evaluation and testing of the attention based model and KNN. The first is a large data set containing samples from 59 classes of varying

0	
0.05	
0.1	0
0.15	0.01
0.2	0.02
0.25	0.03
0.3	0.04
0.35	0.05
0.45	(b) Probabilities of spurious pulses
0.5	

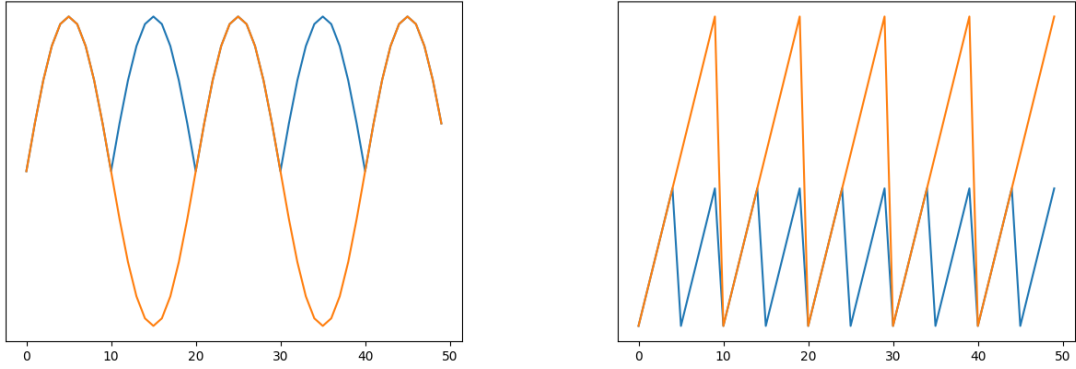
(a) Probabilities of dropped pulses

Table 3.1.7: Spurious and dropped pulse probabilities

Dwell and switch	Fix	Continuous Jitter	Discrete Jitter	Stagger	Sine
0	0	0	0	0	0
0.005	0.1	0	0.1	0.1	0.1
0.01	0.2	0	0.2	0.2	0.2
0.015	0.3	0	0.25	0.25	0.25
0.02	0.4	0	0.3	0.3	0.3
0.025	0.5	0	0.35	0.35	0.35
0.03	0.6	0	0.4	0.4	0.4
0.035	0.7	0	0.5	0.5	0.5
0.04	0.8	0	0.6	0.6	0.6
0.045	0.9	0	0.8	0.8	0.8
0.05	1.0	0	1	1	1.0

Table 3.1.8: Levels of gaussian noise per modulation type

modulation type as per the specifications found in documents supplied by Saab. The intent is to generate data that is as close to the data that is found in real-world scenarios. Noise is injected in the large data set according to the recommendations from experts at Saab. Examples of samples from each class is presented in figure ???. Following this subsection, references to the "large data set" will refer to aforementioned generated data. The second data set is designed to be more difficult than the large data set. This is done by specifying classes that are artificially closer to each other in a euclidean sense than what can be found in the documents supplied by Saab. Some classes are defined as subsets of another class. Examples of this can be seen in figures 3.1.1a and 3.1.1b. Examples of samples from each class is presented in figure 3.1.2b. Following this subsection, references to the "difficult data set" will refer to aforementioned generated data. Two versions of each data set are generated, versions for training and development are generated using the numpy random seed 42 and versions for testing are generated using the numpy random seed 48.



(a) Two classes of modulation type sine

(b) Two classes of modulation type stagger

Figure 3.1.1: Examples of classes that are subsets of other classes in the difficult data set

Algorithm 1 Data generation algorithm

Input: *Library* (Where *Library* is an object containing information of classes and noise)

Output: *Data* (Where $Data = (Class_1, Class_2, \dots, Class_i)$)

Data

```

for Spur in Library(Noisespur) do
  for Drop in Library(Noisedrop) do
    for Gauss in Library(Noisegauss) do
      for Class in Library(Classes) do
         $Data \leftarrow Data + \text{getSamples}(\text{Class}, \text{Spur}, \text{Drop}, \text{Gauss})$ 
      end for
    end for
  end for
end for

```

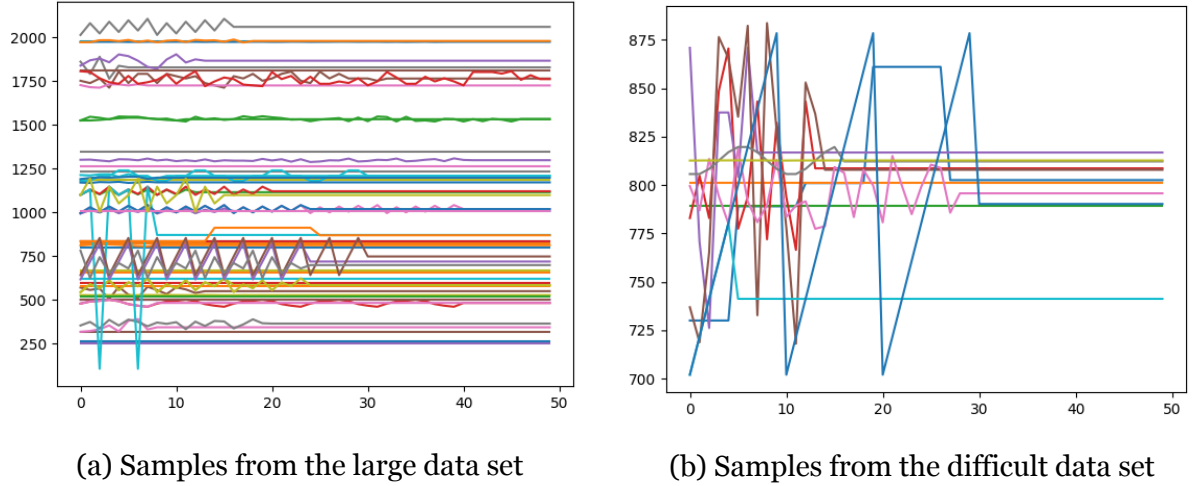


Figure 3.1.2: Graphs of samples from the large and difficult data sets without noise

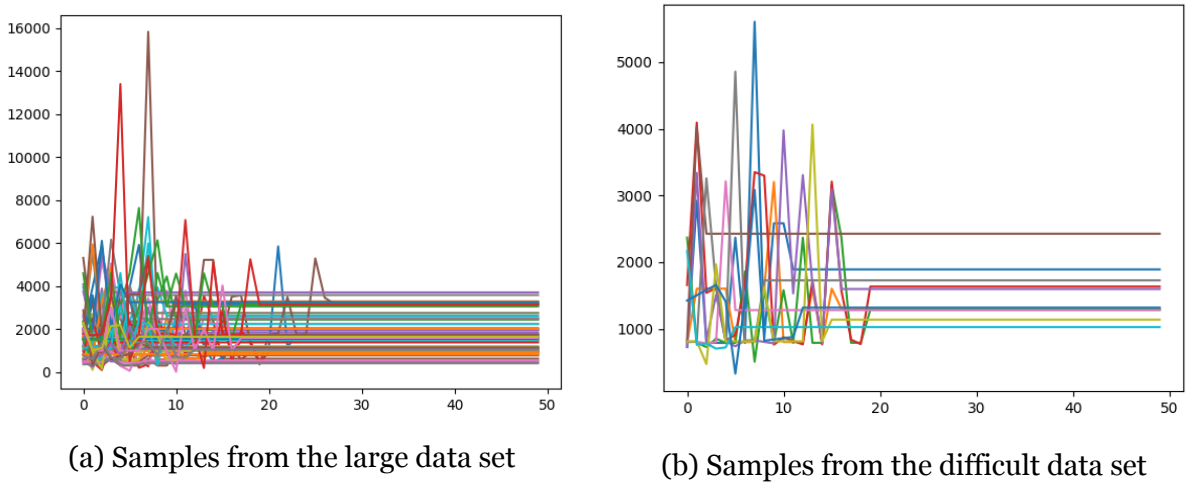


Figure 3.1.3: Graphs of samples from the large and difficult data sets with noise

3.2 Attention Based Model Implementation

Multiple versions of the transformer model with varying parameters and compositions were implemented and tested using the simple data set to gauge viability and use for deciding the final implementation. For instance, the time series data pre-processing step Time2Vec was considered and the option of not performing any transformations on the input. The finalized model implementation consists of an embedding layer and an encoder block as described in the proposed original implementation [34], with a multilayer perceptron of a single hidden layer with a softmax layer at the output for class probability inference.

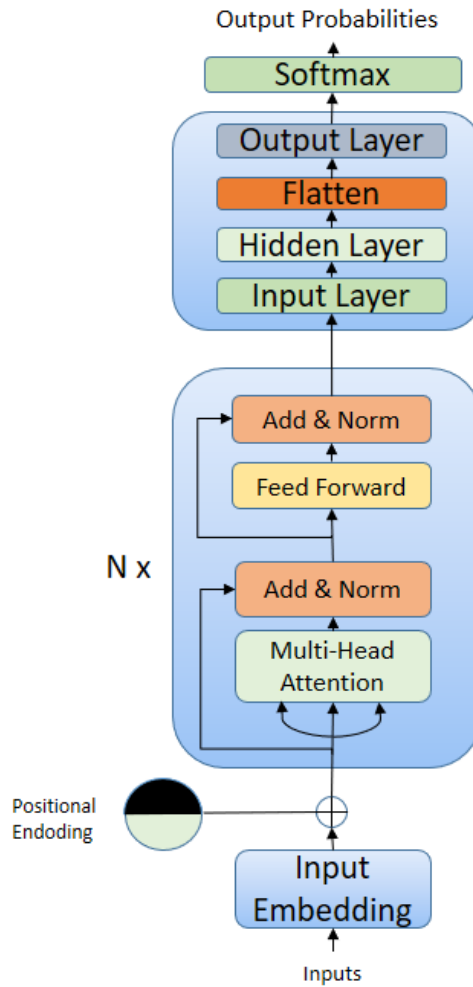


Figure 3.2.1: The architecture of the attention based model

3.2.1 Transformer Encoder

The architecture of the encoder block is illustrated in figure 3.2.1 and consists of a Multi-Head Attention block, a residual connection and Linear Normalization layer, a fully connected feed-forward layer, and an additional residual connection and Linear Normalization Layer.

Embedding In order to test which of the embedding, Time2Vec or no-preprocessing blocks are the best option for the finalized attention based model each version of the architecture is trained and their respective validation accuracy and training loss are compared and evaluated to find the best method for pre-processing the data before input to the network. The embedding step is performed in the same manner as in the original transformer implementation, however, with an additional step that rounds

the input to the closest full integer. This is done to allow for the embedding module to function properly without making the embedding dimension too large. Training is done on the large training set for 10 epochs per version. The architecture for these tests consists of 5 attention heads per multi-head attention layer and with 6 encoder blocks as per the original transformer implementation [34].

Blocks and Attention Heads Tests for the number of attention heads and number of encoder blocks are conducted using the larger, 59 class dataset where the time for training and validation accuracy are the results of interest. This is done in order to gauge how a deeper and wider network performs and decide on the number of attention heads and encoder blocks that should be used in the model. The tests are conducted by training and testing for each combination of number of attention heads and encoder blocks. The training is performed for 2 epochs per configuration.

Multi-Head Attention In the original transformer architecture, the attention heads are fed three different matrices, the keys, queries, values and masked values [34]. However, in the attention based model used in this thesis, the keys, queries and values are all copies of the same matrix; being the embedded samples from the previous embedding block.

Add & Norm The residual connection and layer normalization (addNorm) block used in this thesis is the same as proposed in the original transformer architecture [34]. The residual connection concatenates the embedded data with the output of the Layer Normalization step. The Layer Normalization step consists of two tensors, alpha and beta, of the chosen model length comprised of ones and zeros, respectively. A term denoted epsilon is used to avoid division by zero for very small values of the standard deviation calculated from the input.

Fully Connected Feed Forward The feed forward layer consists of two fully connected layers with ReLU activation and a dropout layer. The dropout rate is set to 0.01 for all dropout layers in the encoder.

Schedule The weights of the attention based model are initialized using Xavier initialization and the optimizer used during training is the Adam optimizer with a

learning rate of $lr = 0.0001$, $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$ [11], [19]. The loss is calculated using the cross entropy loss function [39].

3.2.2 Multi Layer Perceptron

The multi layer perceptron (MLP) consists of an input layer and a hidden layer, both with ReLU as their activation function followed by flattening of the data which is then fed into the output layer which reduces the dimensions of its input to the number of classes for the output. Finally, softmax is applied to the output from the last layer to get the probabilities for each class per sample.

3.3 K-Nearest Neighbor

The KNN implementation uses DTW as a measurement for similarity and LBK to filter the potential matches. The pseudocode for KNN is presented in algorithm 2, DTW in algorithm 3 and LBK in algorithm 4. Testing of the KNN is done with $K = 3$ on the large and difficult data set.

Algorithm 2 K-Nearest Neighbor algorithm

Input: $Train, Test, K$ (Where $Train = (X_1, X_2, \dots, X_s)$ and $Test = (Y_1, Y_2, \dots, Y_t)$)

Output: $Preds$ (Where $Preds = (Z_1, Z_2, \dots, Z_t)$)

```

for  $Y_i, i$  in  $Test$  do
     $dist_{min} \leftarrow inf$ 
    for  $X_j, j$  in  $Train$  do
        if  $getLBK(Y_{i:-1}, X_{j:-1}, U) < dist_{min}$  then
             $dist = getDTW(Y_{i:-1}, X_{j:-1}, K)$ 
            if  $dist < dist_{min}$  then
                 $dist_{min} \leftarrow dist$ 
                 $Z_j \leftarrow X_j$ 
            end if
        end if
    end for
     $Preds \leftarrow Preds \cup Z_{j-1}$ 
end for
end for

```

3.4 Performance metrics

In addition to presenting a clear final accuracy on the generated test data, confusion matrices for the attention based model and KNN for each of the data sets will be

Algorithm 3 Dynamic Time Warping algorithm

Input: X, Y, w (where $X = (x_1, x_2, \dots, x_m)$ and $Y = (y_1, y_2, \dots, y_n)$)**Output:** $\sqrt{DTW_{m-1, n-1}}$ $w \leftarrow \max(w, \text{abs}(m - n))$ **for** i **in** reverse range(m) **do****for** j **in** reverse range(n) **do** $DTW_{i,j} \leftarrow \text{inf}$ **end for****end for** $DTW_{-1,-1} \leftarrow 0$ **for** i **in** range(m) **do****for** j **in** range(n) **do** $d \leftarrow (X_i - Y_j)^2$ $DTW_{i,j} = d + \min(DTW_{i-1,j}, DTW_{i,j-1}, DTW_{i-1,j-1})$ **end for****end for****return** $\sqrt{DTW_{m-1, n-1}}$

Algorithm 4 Lower Bound Keogh algorithm

Input: X, Y, r (where $X = (x_1, x_2, \dots, x_m)$ and $Y = (y_1, y_2, \dots, y_n)$)**Output:** $\sqrt{LB_{sum}}$ $LB_{sum} \leftarrow 0$ **for** x_i, i **in** X **do****if** $x_i - r \geq 0$ **then** $a \leftarrow x_i - r$ **else** $a \leftarrow 0$ **end if** $b \leftarrow x_i + r$ $\text{lower}_{bound} \leftarrow \min(Y_{a:b})$ $\text{upper}_{bound} \leftarrow \max(Y_{a:b})$ **if** $i > \text{upper}_{bound}$ **then** $LB_{sum} \leftarrow LB_{sum} + (i - \text{upper}_{bound})^2$ **else if** $i < \text{lower}_{bound}$ **then** $LB_{sum} \leftarrow LB_{sum} + (i - \text{lower}_{bound})^2$ **end if****end for****return** $\sqrt{LB_{sum}}$

X	0.4	0.6
Y	0.2	0.8
	X	Y

Table 3.4.1: Example of a confusion matrix with labels on both axes and error rates in the cells

presented. This is done in order to investigate whether the attention based model and KNN are able to differentiate between classes accurately. The confusion matrix shows the relation between predicted labels and true labels for each data set [27]. Table 3.4.1 illustrates how a confusion matrix is constructed where one can infer that for 40% of predictions of class X are correct and 60% of predictions are incorrectly predicted to be of class Y . This allows for insight into what classes might be more difficult for the model to accurately predict. Additionally, metrics such as precision, recall, f1-score will be presented for the attention based model and KNN for each of the data sets. Precision is metric for measuring how many positive predictions are correct. It is calculated by dividing the number of predicted true positives with the number of true positives and true negatives as can be seen in equation 3.1. Recall is a metric for measuring how many true positives the that were correctly predicted for all true positives. The calculation is seen in equation 3.2. Combining the previous metrics, F1-Score measures the relationship between precision and recall returns a ratio of the two. The F1-score gives insight in regards to how balanced the predictions are [27]. The calculation is shown in equation 3.3.

$$Precision = \frac{Predicted\ True\ Positives}{True\ Positives + True\ Negatives} \quad (3.1)$$

$$Recall = \frac{Predicted\ true\ positives}{True\ Positives + False\ Negatives} \quad (3.2)$$

$$F1-Score = 2 \cdot \frac{Precision + Recall}{Precision \cdot Recall} \quad (3.3)$$

3.5 Hardware and Software

PyTorch is used for the development and implementation of the attention based model. PyTorch is an open source machine learning framework based on the Torch

library, originally developed using the Lua programming language. PyTorch is as of today actively being developed by Meta (Facebook). The PyTorch framework allows for computations on CUDA-capable Nvidia GPU:s which allows for faster operations and matrix calculations reducing training times of PyTorch-implemented models significantly [26]. The KNN is implemented in the python language using numpy and scipy [16] [35]. The evaluation and testing of the attention based model and KN are done using the computational cluster on Saab and using an NVIDIA Quadro 5000 GPU during training and development of the attention based model.

Chapter 4

Results and Analysis

In the first section of this chapter the results from the evaluation of the different attention based model architectures will be presented. The sections following will be presenting the confusion matrices and multiple metrics from the testing of the finalized model architecture based on previous results and the results from the same tests using the KNN. It is important to take note of the colorbar next to each confusion matrix since, for example, a cell colored in a dark blue could represent a different value for each matrix.

4.1 Attention model architecture

4.1.1 Embedding the inputs leads to better predictions

Figures 4.1.1a and 4.1.1b shows the results obtained from training three attention models for 10 epochs using three different embedding blocks; with embedding, without embedding, and using time2Vec. The graph in figure 4.1.1a presents the validation accuracy of the three models, and figure 4.1.1b present the training loss. The results shows that the model using the embedding is the better in regards to both metrics, with highest validation accuracy and lowest training loss. The following results in this chapter are therefore obtained from an attention model using the embedding block.

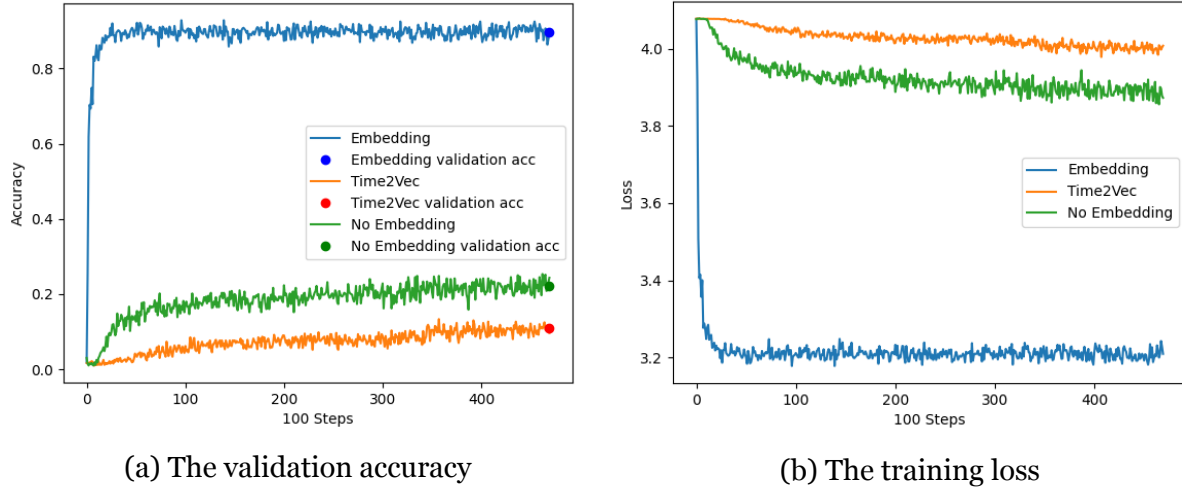


Figure 4.1.1: The validation accuracy (a) and training loss (b) for three models using embedding, without embedding and Time2Vec

4.1.2 Increasing the number of attention heads and encoder blocks does not improve predictive qualities

To determine the final design of the model architecture, the training time and validation accuracy of the model was tested further using different combinations of attention heads and encoding blocks. Table 4.1.1 shows that the validation accuracy and the training time is decreased as the network increases in size and width. The five best training times and validation accuracies from this test is shaded in gray. The model that performs the best in terms of validation accuracy and training time is the architecture using one attention head and one encoder. The following results are therefore generated using a model with one attention head per multi-head attention block and a single encoder block.

4.2 The attention based model is a good predictor for multiple classes

In order to test the model's capability of differentiating between samples from different classes of the same modulation type, confusion matrices were created for the large and difficult data set shown in figures 4.2.1 and 4.2.2. Figure 4.2.1 shows the confusion matrix for the large data set of 59 classes. The result shows that the attention based model is accurate in its predictions for most classes in the data. Some outliers are classes with the continuous jitter as their modulation type for which the model shows

Heads	Encoders	Time	Accuracy
1	1	8:22	0.891580
1	2	11:51	0.891500
1	3	15:30	0.862369
1	4	19:16	0.886059
2	1	8:24	0.855487
2	2	12:10	0.877265
2	3	19:52	0.86562
2	4	24:31	0.88220
5	1	8:45	0.873823
5	2	12:53	0.871733
5	3	21:04	0.874335
5	4	26:23	0.881752
10	1	9:20	0.875550
10	2	14:05	0.892675
10	3	18:55	0.872495
10	4	23:22	0.873059
25	1	11:10	0.873626
25	2	17:41	0.865242
25	3	24:22	0.865781
25	4	32:04	0.841150

Table 4.1.1: The training times and validation accuracies for the different combinations of attention heads and encoders for 2 epochs of training. The cells with the five highest accuracies and lowest training times are shaded grey.

a lesser predictive ability. The confusion matrix for the difficult data set of 11 classes are shown figure 4.2.2. The result from this figure shows that the model is capable of giving accurate predictions when samples have are close in their mean values and are similar in operation. Outliers in this figure are the classes with sine, discrete jitter and continuous jitter as their modulation types. Tables A.2.1 and A.2.3 show the precision, recall and f1-score for the large and difficult data set respectively.

4.3 K-Nearest Neighbor is a limited predictor for multiple classes

The confusion matrices for the tests of KNN on the large and difficult data sets can be seen in figure 4.3.1 and 4.2.2 respectively. The KNN is limited in its ability to generate accurate predictions in the large data set and is severely limited in its predictive capabilities in regards to its performance during tests on the difficult data set. Tables A.2.4 and A.2.6 shows the precision, recall and f1-score for the large and difficult data

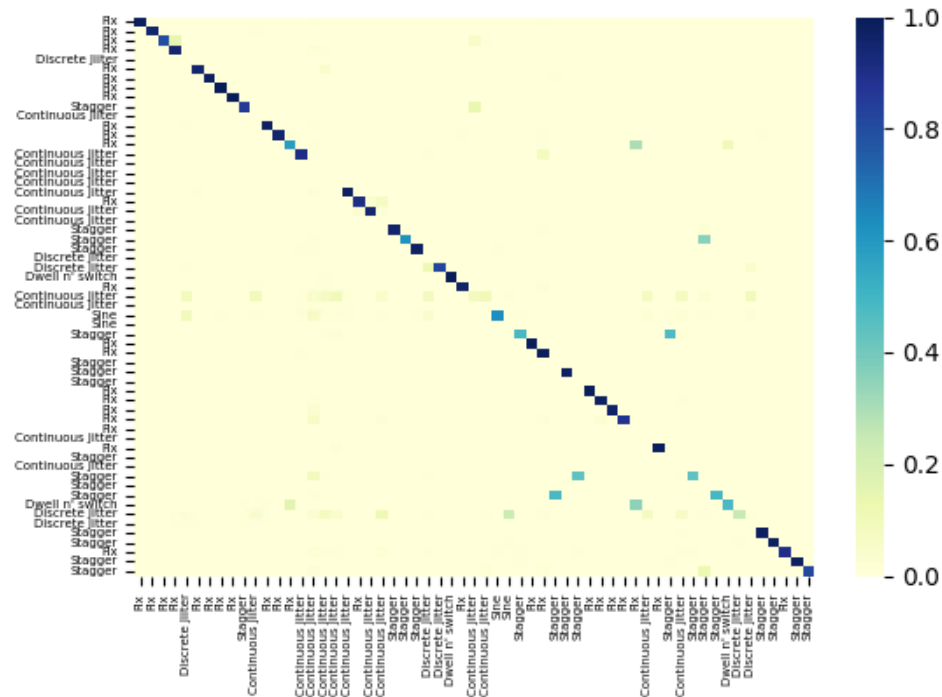


Figure 4.2.1: The confusion matrix of the attention based model on the new large data set

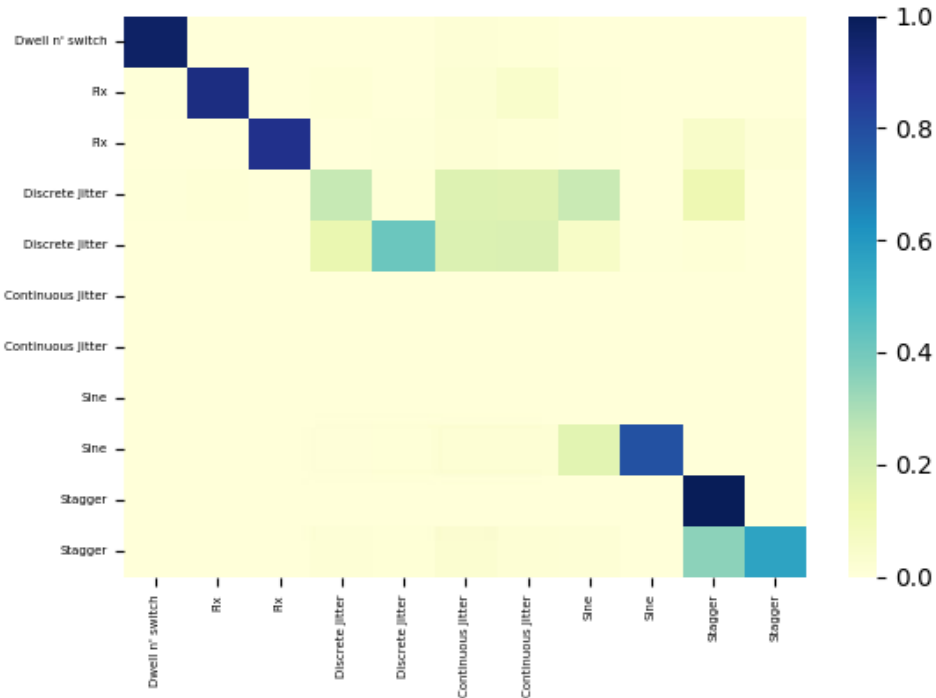


Figure 4.2.2: The confusion matrix of the attention based model on the new difficult data set

set respectively.

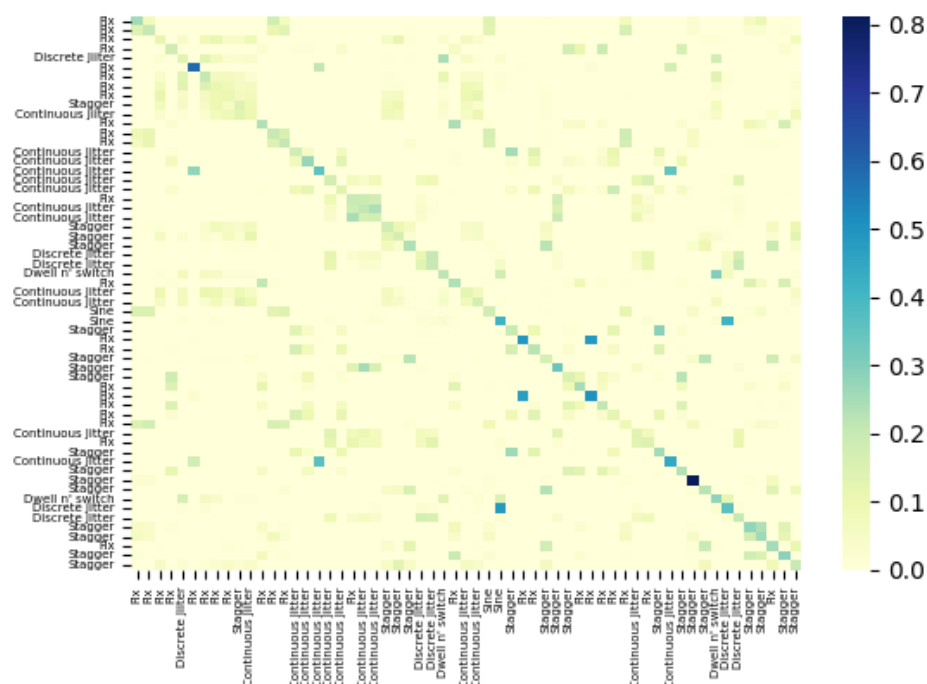


Figure 4.3.1: The confusion matrix of the KNN on the new large data set

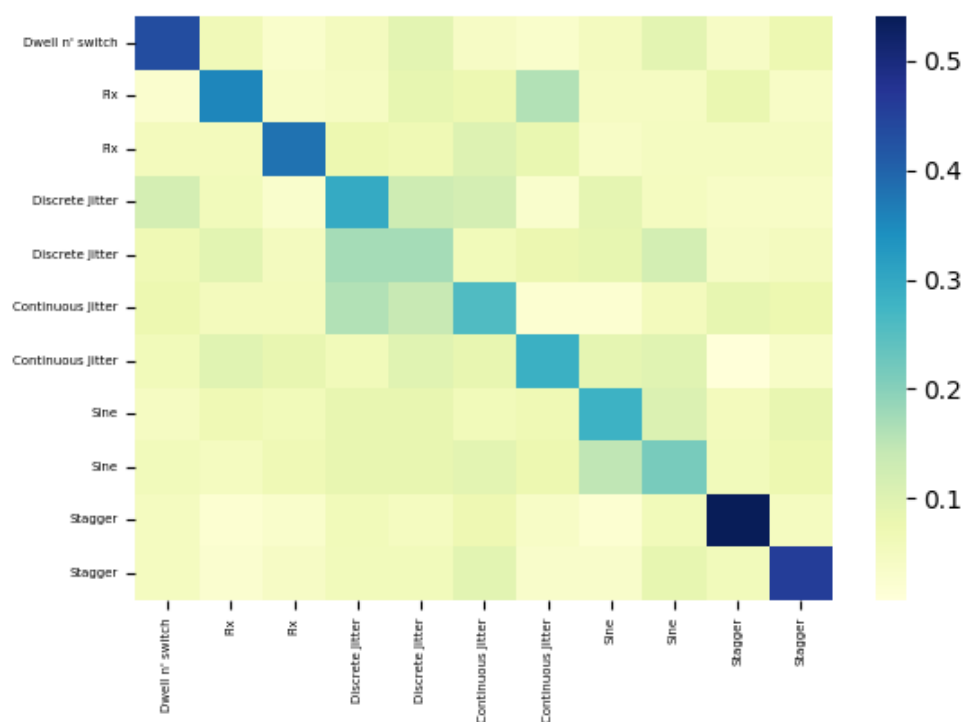


Figure 4.3.2: The confusion matrix of the KNN on the new difficult data set

Chapter 5

Discussion

5.1 The role of the embedding block

While the results show that the embedding block used in the attention based model is necessary for good results, it could possibly be removed or changed in order to more accurately process the information of the input. The embedding of values should in theory not have an impact on the attention based models performance in regards to learning but it can be theorized that the embedding step produces representations that are easier for the model to garner feature knowledge from. As presented in chapter 2, figure 2.3.2, the input values are transformed to vector representations comprised of relatively small values. For example, say that the value 1511.012 is transformed in the embedding block to the vector representation 0.152, -1.234, 0, 991, the smaller values could lead to a better, unique chain of unit activation in the model leading to a better tuned model. One could also compare the embedding step to the Fourier transform where transformation of signal data can lead to better understanding of the inherent features.

5.2 The impact of attention heads and encoder blocks

One interesting result is that of table 4.1.1 where it is shown that increasing the model complexity leads to a worse or equally performing network in regards to the validation accuracy and time spent training. This phenomenon could most likely be

attributed to that as model complexity increases, the chance of overfitting increases. However, this is not an undesirable outcome, on the contrary, having a relatively small model that is quick to train is very beneficial in the domain of EW. For instance, due to the implementation being quick to train, new, unseen data can be added to the existing library and be used to improve the generalization capabilities of the model. Additionally, a point of interest is that the network is quick to converge during training leading to a small amount of epochs required to get accurate results from new data.

5.3 The strength and weaknesses of the attention based model

The confusion matrices found in figure 4.2.1 and 4.2.2 from the testing of the attention based model show that the model is very accurate in its predictions and can, for the most part, differentiate between emitter realizations of the same modulation type. However, there are some emitter classes that the models struggles with. For instance, in table A.2.1 there are classes that the model is unable to accurately classify and for some classes the model is unable to give a correct prediction at all. The most common modulation type of the classes that the model is unable to correctly classify is "continuous jitter". This is not entirely unexpected since the modulation type is defined as being stochastic in nature, therefore the only unique feature of this modulation type is the mean and standard deviation. However, due to the padding the original mean and standard deviation can be lost and lead to samples of different classes to be too close in a euclidean sense. The classes and their respective precision, recall and F1-score are found in table A.2.1. The class at index 44 of the table is defined as having modulation type "fix" and with a constant PRI value of $835.2\mu s$. The reason that it could be difficult for the transformer to distinguish this emitter from others is that multiple emitters can be very similar, meaning that they have the same modulation type, therefore having the same levels of noise as described in table 3.1.8, and having a relatively small difference in PRI value.

5.4 The struggling K-nearest neighbor

The results also show that the predictive capabilities of the KNN are adequate for the large data set as seen in figure 4.3.1, the performance degrades substantially for the difficult data set as seen in figure 4.3.2. The loss in performance for the difficult set is not unexpected since the main focus of the defining the difficult set was to define emitters that were close in distance. However, the KNN performance on the large data set is also relatively poor compared to the attention based model. Even though some classes of the data should be distinguishable by their mean PRI value 3.1.2a, this does not seem to be reflected in the KNN performance 4.3.1. Therefore, it is safe to assume that the noise in the data makes it more complex than what the KNN implementation is capable of classifying correctly.

5.5 Generating large amounts of data

A large portion of the time spent working on this thesis project has been put on writing scripts for the sake of data generation. Although comparatively small parts of the data generation scripts were supplied by Saab, most of it has been developed during this thesis. The final version of the data generation script contains 555 lines of code, illustrating the complexity of generating PRI data and the development time of the script. The files containing the emitter definitions were written following the definitions found in the Saab and FRA documents which were found to have overlap for some emitters after closer inspection of the first round of confusion matrices. Since all data used in this thesis work are artificially generated, it is not possible to accurately predict how the methods would perform in the task of PRI classification on real world data. The results of this thesis can therefore only indicate as to how the methods would perform when applied to real world data. Additionally, there could also be a number of discrepancies between the generated data and the real world data. For instance, like mentioned in subsection 2.1.1, the real world data could be more complex, containing features such as pulse width and bandwidth. Features that have not been taken into account in this thesis project. Furthermore, due to continued development of emitter systems, the PRI found in real world scenarios could have become more complex in their operations since the Saab and FRA documents were created, rendering the emitter definitions found in the documents outdated. This thesis also assumes that there is constant levels of noise in the data which in the real world, might not always

be the case. As mentioned in subsection 2.1.1, noise is not inherently part of the PRI sequence but rather a consequence of the data collection. This means that the presence of noise in real world scenarios could be more or less severe than in the generated data.

5.6 Impact of the mean

In order for the generated data to be as close to the data that can be collected in real world scenarios, samples generated are not always of a fixed size. However, since the attention based model expects an input of a certain length, the samples needs to be padded after generation in order to be of the correct length. The padding used to generate figures A.1.1, A.1.2, A.1.3 and A.1.4 in the appendix, is performed by appending the mean of the values used during generation. For example, a sample of the modulation type "fix" would be padded using the same PRI value used for its generation. The PRI value is unknown during collection and using this value for the padding injects information into the samples that the model would otherwise not be privy to. One can assume that the data is "too easy" and the results in the previously mentioned figures are artificially inflated. Therefore, a second method for generating data is implemented to solve this issue. In the new method the order of data generation is thus; samples are generated, noise is injected and the padding is thereafter performed by appending the mean of the noisy sample. Additionally, the values representing collected PRI always start at index 0, unlike the previous method where the index could be random. This leads to more unique samples in the data and the data should better reflect the information available to the systems in a real world scenario. The impact of the methods for padding on the performance of the attention based model and KNN can be seen by comparing the figures and tables in sections 4.2, 4.3 with the figures and tables found in appendix, section A.1. Unsurprisingly, the performance is for the most part lower when training and testing on the new data but it should be noted that the KNN sees an increase in performance when tested on the difficult data generated using the new method. This could be attributed to the fact that the generated PRI values in the new method always start at index 0, leading to smaller euclidean distance compared to when the PRI values start at a random index.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

The results show that the attention model implementation using the embedding block outperform implementations using Time2Vec and no processing of the input. The results show that for the generated data, the performance of the attention based model does not increase as the number of attention heads and encoder blocks increase. The attention based model is shown to be able to accurately differentiate between classes of similar modes of operation with some outliers for both the large and difficult data set. In this thesis it is also shown that the attention based model outperforms the KNN algorithm across all metrics when applied on the generated data for the task of PRI classification.

6.2 Limitations

The results and conclusions presented in this thesis have been based on the performance of the methods applied on generated data. This fact limits the reproducibility of this thesis since the data was in part generated using scripts supplied by Saab of which details are confidential. For the same reason the code used in this thesis work will not be made publicly available. Attempts at reproducing the results will have to follow the steps and implementations presented in Methods, chapter 3 of this thesis, which could lead to results that are not as comparable as would be desired. While training the attention based model, the GPU cluster would spontaneously end

the process with the prompt "Killed". This led to multiple trials not producing results and since training for a large amount of epochs would lead to this happening more often than not. It is unknown why this error would occur.

6.3 Future work

A number of changes can be done for the sake of exploring the variations in model training times and parameters of the attention based model. For instance, the embedding block can in theory be removed in favor of another method used for time series data. While Time2Vec was shown to have reduced performance compared to not having any pre-processing at all, one could implement a function that is able to generate unique representations for values in the input that mimics the ones returned from the embedding block. The attention based model implemented could also be too large in regards to model parameters and there could be another, more efficient model that could be used to achieve comparable results. Furthermore, one could model the time series data as a Markov process and possibly garner comparable results as the method proposed in this project. Hidden Markov Models (HMM) have shown great results in the domains of speech recognition and time series classification [3] [15]. An HMM ensemble method has been proposed for classifying time series data with results that indicate that the method could be applied in the problem area of this thesis, albeit the findings are based on data featuring two classes which is not entirely applicable in the problem definition of this thesis [3].

6.4 Reflections

6.4.1 Sustainability, Societal and Ethical aspects of thesis

Saab AB has a great interest in the results or insight that can be gained from this thesis. First and foremost Saab wants to know if it is feasible to utilize machine learning methods for the classification of emitters based on their PRI sequences. The results show that there is great potential for a working, practical implementation that can be delivered and improve upon the existing capabilities of their products.

Naturally, there are a number of ethical and societal aspects to take into account whenever research and development is conducted in the domain of electronic warfare.

Saab as a stakeholder wish to, through this thesis and others, explore the capabilities and potential for incorporating machine learning in their already existing products or future products. However, the thesis is conducted with a surveillance perspective in mind meaning that the goal of this work and future works is for the sake of defensive capabilities. This does not guarantee that no part of this thesis or future works will be used for the development of weapons systems but it would be unlikely.

Bibliography

- [1] “A logical calculus of the ideas immanent in nervous activity.” In: *Bulletin of Mathematical Biophysics* 5, pages 115-133 (Dec. 1943). DOI: <https://doi.org/10.1007/BF02478259>.
- [2] Abdul Ghani, Kamaruddin, Dimyati, Kaharudin, Sha’ameri, A.Z., and Nik Daud, Nik Ghazali. “Statistical modelling for missing and spurious pulses in pulse repetition interval (PRI) analysis”. In: 9 (Jan. 2016), pp. 18–27.
- [3] Asadi, Nazanin, Mirzaei, Abdolreza, and Haghshenas, Ehsan. “Creating Discriminative Models for Time Series Classification and Clustering by HMM Ensembles”. In: *IEEE Transactions on Cybernetics* 46.12 (2016), pp. 2899–2910. DOI: 10.1109/TCYB.2015.2492920.
- [4] Ba, Jimmy Lei, Kiros, Jamie Ryan, and Hinton, Geoffrey E. *Layer Normalization*. 2016. DOI: 10.48550/ARXIV.1607.06450. URL: <https://arxiv.org/abs/1607.06450>.
- [5] Barrios, Karolina. *Automation of Pulse Repetition Interval Modulation Classification*. 2021.
- [6] Berndt, Donald J and Clifford, James. “Using dynamic time warping to find patterns in time series.” In: *KDD workshop*. Vol. 10. 16. Seattle, WA, USA: 1994, pp. 359–370.
- [7] Bhargava, Aditya Y. *Grokking Algorithms*. en. Ed. by Jennifer Stout and Damien White. First. Manning, May 2016. ISBN: 9781617292231. URL: <https://www.manning.com/books/grokking-algorithms>.
- [8] Cordonnier, Jean-Baptiste, Loukas, Andreas, and Jaggi, Martin. “Multi-Head Attention: Collaborate Instead of Concatenate”. In: *CoRR* abs/2006.16362 (2020). arXiv: 2006.16362. URL: <https://arxiv.org/abs/2006.16362>.

- [9] Devlin, Jacob, Chang, Ming-Wei, Lee, Kenton, and Toutanova, Kristina. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *CoRR* abs/1810.04805 (2018). arXiv: 1810.04805. URL: <http://arxiv.org/abs/1810.04805>.
- [10] Ghani, Kamaruddin Abdul, Sha’ameri, Ahmad Zuri, Dimyati, Kaharudin, and Daud, Nik Ghazali Nik. “Pulse repetition interval analysis using decimated Walsh-Hadamard transform”. In: *2017 IEEE Radar Conference (RadarConf)*. 2017, pp. 0058–0063. DOI: 10.1109/RADAR.2017.7944171.
- [11] Glorot, Xavier and Bengio, Yoshua. “Understanding the difficulty of training deep feedforward neural networks”. In: *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2010, pp. 249–256.
- [12] Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [13] Hai-Jun, Liu, Liu, Z., Jiang, W.-L, and Zhou, Yuqiu. “Approach based on combination of vector neural networks for emitter identification”. In: *Signal Processing, IET* 4 (May 2010), pp. 137–148. DOI: 10.1049/iet-spr.2008.0247.
- [14] Han, Jin-Woo and Park, Cheong Hee. “A Unified Method for Deinterleaving and PRI Modulation Recognition of Radar Pulses Based on Deep Neural Networks”. In: *IEEE Access* 9 (2021), pp. 89360–89375. DOI: 10.1109/ACCESS.2021.3091309.
- [15] Hanif, Muhammad, Sami, Faiza, Hyder, Mehvish, and Ch, Muhammad. “Hidden Markov Model for Time Series Prediction”. In: *Journal of Asian Scientific Research* 7 (Jan. 2017), pp. 196–205. DOI: 10.18488/journal.2.2017.75.196.205.
- [16] Harris, Charles R., Millman, K. Jarrod, Walt, Stéfan J. van der, Gommers, Ralf, Virtanen, Pauli, Cournapeau, David, Wieser, Eric, Taylor, Julian, Berg, Sebastian, Smith, Nathaniel J., Kern, Robert, Picus, Matti, Hoyer, Stephan, Kerkwijk, Marten H. van, Brett, Matthew, Haldane, Allan, Río, Jaime Fernández del, Wiebe, Mark, Peterson, Pearu, Gérard-Marchant, Pierre, Sheppard, Kevin, Reddy, Tyler, Weckesser, Warren, Abbasi, Hameer, Gohlke, Christoph, and Oliphant, Travis E. “Array programming with NumPy”. In: *Nature* 585.7825

- (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: <https://doi.org/10.1038/s41586-020-2649-2>.
- [17] Kazemi, Seyed Mehran, Goel, Rishab, Eghbali, Sepehr, Ramanan, Janahan, Sahota, Jaspreet, Thakur, Sanjay, Wu, Stella, Smyth, Cathal, Poupart, Pascal, and Brubaker, Marcus A. “Time2Vec: Learning a Vector Representation of Time”. In: *CoRR* abs/1907.05321 (2019). arXiv: 1907.05321. URL: <http://arxiv.org/abs/1907.05321>.
- [18] Keogh, Eamonn and Ratanamahatana, Chotirat Ann. “Exact indexing of dynamic time warping”. en. In: *Knowledge and Information Systems* 7.3 (Mar. 2005), pp. 358–386. ISSN: 0219-3116. DOI: 10.1007/s10115-004-0154-9. URL: <https://doi.org/10.1007/s10115-004-0154-9> (visited on 06/05/2022).
- [19] Kingma, Diederik P and Ba, Jimmy. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [20] Köbis, Nils and Mossink, Luca D. “Artificial intelligence versus Maya Angelou: Experimental evidence that people cannot differentiate AI-generated from human-written poetry”. In: *Computers in Human Behavior* 114 (2021), p. 106553. ISSN: 0747-5632. DOI: <https://doi.org/10.1016/j.chb.2020.106553>. URL: <https://www.sciencedirect.com/science/article/pii/S0747563220303034>.
- [21] Liu, Yinhan, Ott, Myle, Goyal, Naman, Du, Jingfei, Joshi, Mandar, Chen, Danqi, Levy, Omer, Lewis, Mike, Zettlemoyer, Luke, and Stoyanov, Veselin. “RoBERTa: A Robustly Optimized BERT Pretraining Approach”. In: *CoRR* abs/1907.11692 (2019). arXiv: 1907.11692. URL: <http://arxiv.org/abs/1907.11692>.
- [22] Maas, Andrew L, Hannun, Awni Y, Ng, Andrew Y, et al. “Rectifier nonlinearities improve neural network acoustic models”. In: *Proc. icml*. Vol. 30. 1. Citeseer. 2013, p. 3.
- [23] Miah, Tasnia and Bashir, Omar. “An Expert System For Threat Analysis In Radar Warning Receivers”. In: 5 (Jan. 1999).
- [24] Noone, G.P. “A neural approach to automatic pulse repetition interval modulation recognition”. In: *1999 Information, Decision and Control. Data and Information Fusion Symposium, Signal Processing and Communications Symposium and Decision*

- and Control Symposium. Proceedings (Cat. No.99EX251)*. 1999, pp. 213–218. DOI: 10.1109/IDC.1999.754156.
- [25] Norgren, Eric. *Pulse repetition interval modulation classification using machine learning*. 2019.
- [26] Paszke, Adam, Gross, Sam, Massa, Francisco, Lerer, Adam, Bradbury, James, Chanan, Gregory, Killeen, Trevor, Lin, Zeming, Gimelshein, Natalia, Antiga, Luca, Desmaison, Alban, Kopf, Andreas, Yang, Edward, DeVito, Zachary, Raison, Martin, Tejani, Alykhan, Chilamkurthy, Sasank, Steiner, Benoit, Fang, Lu, Bai, Junjie, and Chintala, Soumith. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems 32*. Ed. by H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett. Curran Associates, Inc., 2019, pp. 8024–8035. URL: <http://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [27] Powers, David. “Evaluation: From Precision, Recall and F-Factor to ROC, Informedness, Markedness Correlation”. In: *Mach. Learn. Technol.* 2 (Jan. 2008).
- [28] Prätzlich, Thomas, Driedger, Jonathan, and Müller, Meinard. “Memory-restricted multiscale dynamic time warping”. In: *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2016, pp. 569–573. DOI: 10.1109/ICASSP.2016.7471739.
- [29] Sergei A. Vakin Lev N. Shustov, Robert H. Dunwell. *Fundamentals of Electronic Warfare*. 2001. ISBN: 1-58053-052-4.
- [30] Shi, Zhongya, Wu, Hua, Shen, Wendi, Cheng, Siyi, and Chen, You. “Feature extraction for complicated radar PRI modulation modes based on auto-correlation function”. In: *2016 IEEE Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC)*. Oct. 2016, pp. 1617–1620. DOI: 10.1109/IMCEC.2016.7867491.
- [31] Shieh, Ching-Sung and Lin, Chin-Teng. “A vector neural network for emitter identification”. In: *IEEE Transactions on Antennas and Propagation* 50.8 (2002), pp. 1120–1127. DOI: 10.1109/TAP.2002.801387.
- [32] Skolnik, Merrill I. *Introduction to Radar Systems*. McGraw-Hill Book Company, 1981.

- [33] Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N, Kaiser, Łukasz, and Polosukhin, Illia. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc., 2017. URL: <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
- [34] Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N, Kaiser, Łukasz, and Polosukhin, Illia. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc., 2017. URL: <https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
- [35] Virtanen, Pauli, Gommers, Ralf, Oliphant, Travis E., Haberland, Matt, Reddy, Tyler, Cournapeau, David, Burovski, Evgeni, Peterson, Pearu, Weckesser, Warren, Bright, Jonathan, van der Walt, Stéfan J., Brett, Matthew, Wilson, Joshua, Millman, K. Jarrod, Mayorov, Nikolay, Nelson, Andrew R. J., Jones, Eric, Kern, Robert, Larson, Eric, Carey, C J, Polat, İlhan, Feng, Yu, Moore, Eric W., VanderPlas, Jake, Laxalde, Denis, Perktold, Josef, Cimrman, Robert, Henriksen, Ian, Quintero, E. A., Harris, Charles R., Archibald, Anne M., Ribeiro, Antônio H., Pedregosa, Fabian, van Mulbregt, Paul, and SciPy 1.0 Contributors. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.
- [36] Voita, Elena, Talbot, David, Moiseev, Fedor, Sennrich, Rico, and Titov, Ivan. “Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned”. In: *CoRR abs/1905.09418* (2019). arXiv: 1905.09418. URL: <http://arxiv.org/abs/1905.09418>.
- [37] Wankmüller, Sandra. “Neural Transfer Learning with Transformers for Social Science Text Analysis”. In: *arXiv:2102.02111 [cs, stat]* (Feb. 2021). arXiv: 2102.02111. URL: <http://arxiv.org/abs/2102.02111> (visited on 02/15/2022).

- [38] *What is the k-nearest neighbors algorithm?* Accessed: 2022-06-03. URL: <https://www.ibm.com/topics/knn>.
- [39] Zhang, Zhilu and Sabuncu, Mert. “Generalized cross entropy loss for training deep neural networks with noisy labels”. In: *Advances in neural information processing systems* 31 (2018).

Appendix A

Appendix

A.1 Performance on data generated using the first method

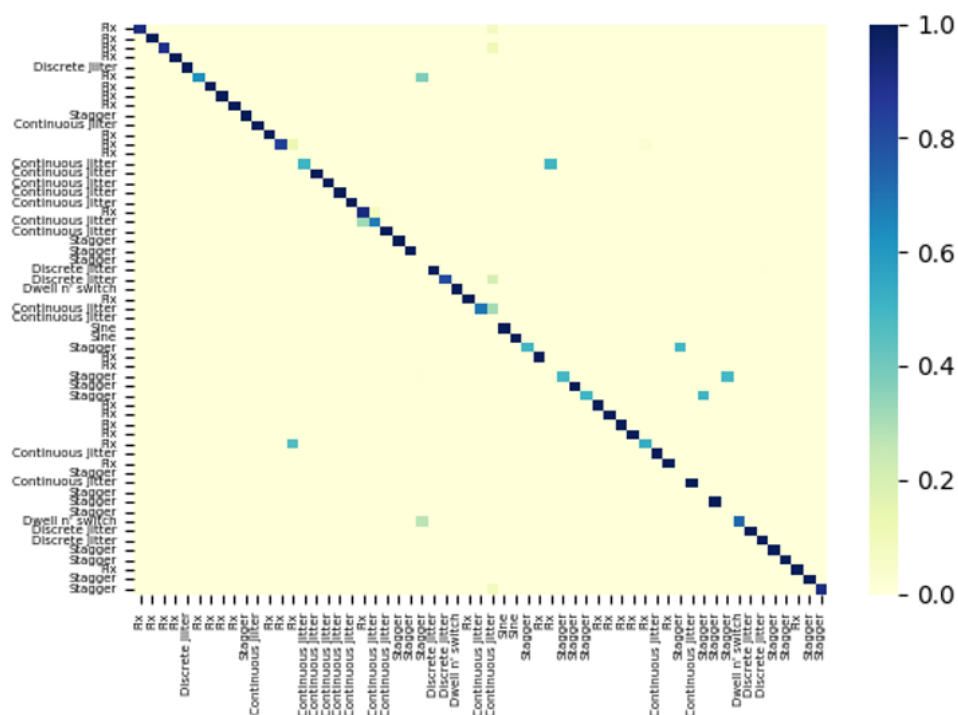


Figure A.1.1: The confusion matrix of the attention based model on the large data set

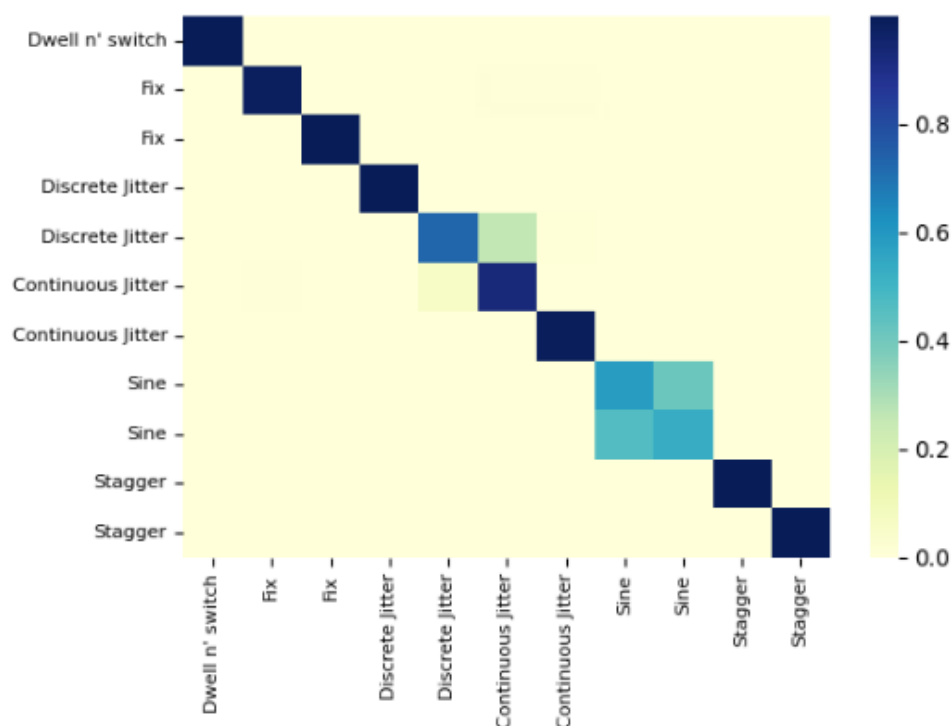


Figure A.1.2: The confusion matrix of the attention based model on the difficult data set

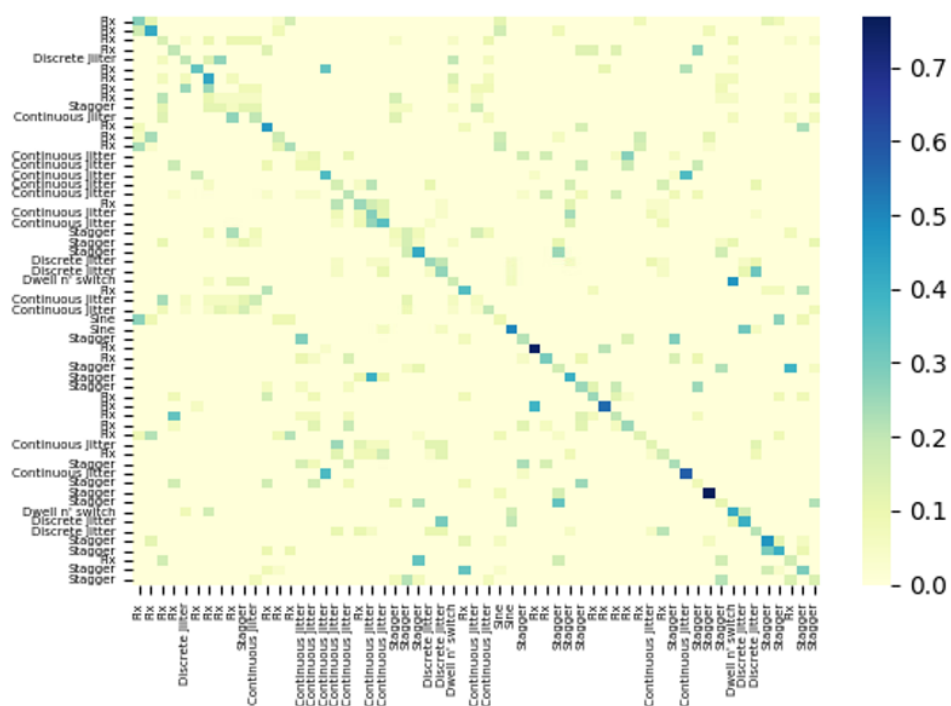


Figure A.1.3: The confusion matrix of the KNN on the large data set

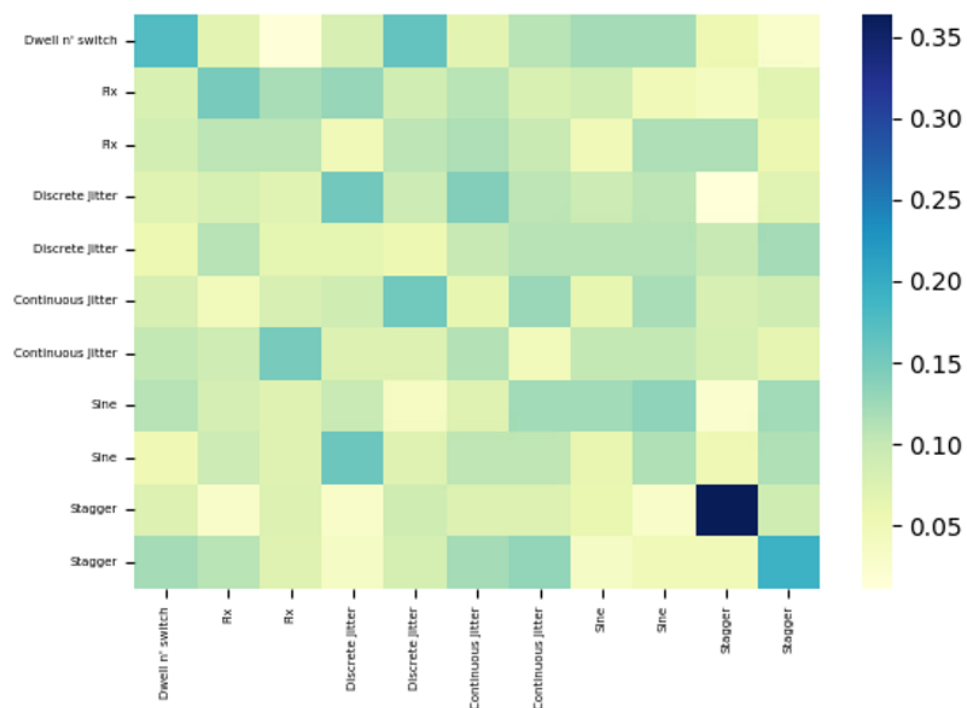


Figure A.1.4: The confusion matrix of the KNN on the difficult data set

Index	Class	Precision	Recall	f1-score	Support
1	Fix	0.28	0.32	0.30	22
2	Fix	0.42	0.24	0.30	21
3	Fix	0.15	0.13	0.14	23
4	Fix	0.20	0.19	0.19	16
5	Discrete Jitter	0.20	0.27	0.23	11
6	Fix	0.33	0.38	0.35	8
7	Fix	0.43	0.25	0.32	24
8	Fix	0.00	0.00	0.00	11
9	Fix	0.05	0.06	0.05	18
10	Stagger	0.12	0.12	0.12	16
11	Continuous Jitter	0.20	0.19	0.19	16
12	Continuous Jitter	0.46	0.26	0.33	23
13	Fix	0.18	0.25	0.21	12
14	Fix	0.23	0.35	0.28	17
15	Fix	0.06	0.08	0.06	13
16	Continuous Jitter	0.09	0.08	0.08	13
17	Continuous Jitter	0.36	0.42	0.39	19
18	Continuous Jitter	0.16	0.14	0.15	22
19	Continuous Jitter	0.21	0.28	0.24	18
20	Continuous Jitter	0.25	0.33	0.29	18
21	Fix	0.28	0.23	0.25	30
22	Continuous Jitter	0.35	0.33	0.34	24
23	Continuous Jitter	0.08	0.09	0.08	11
24	Stagger	0.16	0.19	0.17	16
25	Stagger	0.42	0.29	0.34	17
26	Stagger	0.25	0.36	0.29	14
27	Discrete Jitter	0.26	0.23	0.24	22
28	Dwell n' Switch	0.18	0.21	0.19	14
29	Fix	0.36	0.25	0.29	20
30	Continuous Jitter	0.06	0.11	0.08	9

Table A.1.1: Precision, recall, f1-score and number of samples per class for the KNN on the old large data set

Index	Class	Precision	Recall	f1-score	Support
31	Continuous Jitter	0.20	0.38	0.26	13
32	Sine	0.09	0.05	0.07	19
33	Sine	0.50	0.53	0.52	15
34	Stagger	0.21	0.21	0.21	14
35	Fix	0.75	0.69	0.72	26
36	Fix	0.30	0.38	0.33	16
37	Stagger	0.17	0.17	0.17	18
38	Stagger	0.40	0.17	0.24	23
39	Stagger	0.25	0.19	0.22	21
40	Fix	0.25	0.50	0.33	12
41	Fix	0.56	0.62	0.59	16
42	Fix	0.20	0.18	0.19	17
43	Fix	0.25	0.29	0.27	14
44	Fix	0.13	0.25	0.17	12
45	Continuous Jitter	0.12	0.22	0.16	9
46	Fix	0.17	0.24	0.20	17
47	Stagger	0.23	0.21	0.22	14
48	Continuous Jitter	0.58	0.52	0.55	21
49	Stagger	0.08	0.06	0.07	16
50	Stagger	0.77	0.67	0.71	15
51	Stagger	0.11	0.06	0.08	16
52	Dwell n' Switch	0.42	0.23	0.29	22
53	Discrete Jitter	0.40	0.29	0.33	14
54	Discrete Jitter	0.21	0.20	0.21	20
55	Stagger	0.47	0.42	0.44	19
56	Stagger	0.40	0.24	0.30	17
57	Fix	0.17	0.13	0.15	15
58	Stagger	0.29	0.39	0.33	18
59	Stagger	0.14	0.15	0.15	13
Accuracy				0.26	1000
Macro avg		0.26	0.26	0.25	1000
Weighted avg		0.28	0.26	0.27	1000

Table A.1.2: Continuation of table A.1.1. Precision, recall, f1-score and number of samples per class for the KNN on the old large data set

index	Class	Precision	Recall	f1-score	Support
1	Dwell n' Switch	0.26	0.21	0.23	119
2	Fix	0.14	0.13	0.14	128
3	Fix	0.12	0.14	0.13	122
4	Discrete Jitter	0.10	0.10	0.10	126
5	Discrete Jitter	0.08	0.08	0.08	123
6	Continuous Jitter	0.12	0.14	0.13	111
7	Continuous Jitter	0.10	0.11	0.10	121
8	Sine	0.08	0.08	0.08	110
9	Sine	0.10	0.11	0.10	110
10	Stagger	0.31	0.25	0.27	125
11	Stagger	0.20	0.21	0.21	110
Accuracy				0.14	1305
Macro avg		0.14	0.14	0.14	1305
Weighted avg		0.14	0.14	0.14	1305

Table A.1.3: The precision, recall, f1-score and number of samples per class for the KNN on the old difficult dataset

Index	Class	Precision	Recall	F1-score	support
1	Fix	0.91	1.00	0.96	59400
2	Fix	1.00	1.00	1.00	59400
3	Fix	0.90	1.00	0.95	59400
4	Fix	1.00	1.00	1.00	59400
5	Discrete Jitter	1.00	1.00	1.00	59400
6	Fix	0.62	1.00	0.77	59400
7	Fix	1.00	1.00	1.00	59400
8	Fix	1.00	1.00	1.00	59400
9	Fix	1.00	1.00	1.00	59400
10	Stagger	1.00	1.00	1.00	59400
11	Continuous Jitter	1.00	1.00	1.00	59400
12	Continuous Jitter	1.00	1.00	1.00	59400
13	Fix	0.84	1.00	0.91	59400
14	Fix	0.00	0.00	0.00	59400
15	Fix	0.50	1.00	0.67	59400
16	Continuous Jitter	1.00	1.00	1.00	59400
17	Continuous Jitter	1.00	1.00	1.00	59400
18	Continuous Jitter	1.00	1.00	1.00	59400
19	Continuous Jitter	1.00	1.00	1.00	59400
20	Continuous Jitter	0.92	0.55	0.69	59400
21	Fix	0.68	0.95	0.79	59400
22	Continuous Jitter	1.00	1.00	1.00	59400
23	Continuous Jitter	1.00	1.00	1.00	59400
24	Stagger	1.00	1.00	1.00	59400
25	Stagger	0.00	0.00	0.00	59400
26	Stagger	1.00	1.00	1.00	59400
27	Discrete Jitter	0.80	1.00	0.89	59400
28	Dwell n' Switch	1.00	1.00	1.00	59400
29	Fix	1.00	1.00	1.00	59400
30	Continuous Jitter	0.69	1.00	0.82	59400

Table A.1.4: Precision, recall, f1-score and number of samples per class for the KNN on the old large data set

Index	Class	Precision	Recall	F1-score	support
31	Continuous Jitter	0.00	0.00	0.00	59400
32	Sine	1.00	1.00	1.00	59400
33	Siner	1.00	1.00	1.00	59400
34	Stagger	0.50	1.00	0.67	59400
35	Fix	1.00	1.00	1.00	59400
36	Fix	0.00	0.00	0.00	59400
37	Stagger	0.49	1.00	0.66	59400
38	Stagger	1.00	1.00	1.00	59400
39	Stagger	0.50	1.00	0.67	59400
40	Fix	1.00	1.00	1.00	59400
41	Fix	1.00	1.00	1.00	59400
42	Fix	1.00	1.00	1.00	59400
43	Fix	1.00	1.00	1.00	59400
44	Fix	0.53	0.97	0.69	59400
45	Continuous Jitter	1.00	1.00	1.00	59400
46	Fix	1.00	1.00	1.00	59400
47	Stagger	0.00	0.00	0.00	59400
48	Continuous Jitter	1.00	1.00	1.00	59400
49	Stagger	0.00	0.00	0.00	59400
50	Stagger	1.00	1.00	1.00	59400
51	Stagger	0.00	0.00	0.00	59400
52	Dwell n' Switch	0.73	1.00	0.84	59400
53	Discrete Jitter	1.00	1.00	1.00	59400
54	Discrete Jitter	1.00	1.00	1.00	59400
55	Stagger	1.00	1.00	1.00	59400
56	Stagger	1.00	1.00	1.00	59400
57	Fix	1.00	1.00	1.00	59400
58	Stagger	1.00	1.00	1.00	59400
59	Stagger	0.91	1.00	0.95	59400
Accuracy				0.87	3504600
Macro avg		0.81	0.87	0.83	3504600
Weighted avg		0.81	0.87	0.83	3504600

Table A.1.5: Continuation of table A.1.4. Precision, recall, f1-score and number of samples per class for the attention based model on the old large data set

Index	Class	Precision	Recall	F1-score	Support
1	Dwell n' Switch	1.00	1.00	1.00	59400
2	Fix	0.98	0.99	0.99	59400
3	Fix	1.00	1.00	1.00	59400
4	Discrete Jitter	1.00	1.00	1.00	59400
5	Discrete Jitter	0.73	0.95	0.83	59400
6	Continuous Jitter	0.93	0.65	0.77	59400
7	Continuous Jitter	0.99	0.98	0.99	59400
8	Sine	0.58	0.33	0.42	59400
9	Sine	0.53	0.77	0.63	59400
10	Stagger	1.00	1.00	1.00	59400
11	Stagger	1.00	1.00	1.00	59400
Accuracy				0.88	653400
Macro avg		0.89	0.88	0.87	653400
Weighted avg		0.89	0.88	0.87	653400

Table A.1.6: he precision, recall, f1-score and number of samples per class for the attention based model on the old difficult dataset

A.2 Performance on data generated using the second method

Index	Class	Precision	Recall	F1-score	support
1	Fix	0.98	0.99	0.99	594
2	Fix	0.95	0.99	0.97	594
3	Fix	0.80	0.99	0.89	594
4	Fix	0.95	0.82	0.88	594
5	Discrete Jitter	0.00	0.00	0.00	594
6	Fix	0.95	0.98	0.97	594
7	Fix	0.98	0.99	0.99	594
8	Fix	1.00	0.97	0.99	594
9	Fix	1.00	0.98	0.99	594
10	Stagger	0.86	1.00	0.92	594
11	Continuous Jitter	0.00	0.00	0.00	594
12	Continuous Jitter	0.98	0.99	0.99	594
13	Fix	0.96	0.99	0.97	594
14	Fix	0.59	0.70	0.64	594
15	Fix	0.90	0.96	0.93	594
16	Continuous Jitter	0.00	0.00	0.00	594
17	Continuous Jitter	0.00	0.00	0.00	594
18	Continuous Jitter	0.00	0.00	0.00	594
19	Continuous Jitter	0.98	0.99	0.98	594
20	Continuous Jitter	0.90	0.97	0.93	594
21	Fix	0.95	0.96	0.95	594
22	Continuous Jitter	0.00	0.00	0.00	594
23	Continuous Jitter	0.96	0.99	0.97	594
24	Stagger	0.62	0.99	0.77	594
25	Stagger	0.97	0.97	0.97	594
26	Stagger	0.00	0.00	0.00	594
27	Discrete Jitter	0.82	1.00	0.90	594
28	Dwell n' Switch	1.00	1.00	1.00	594
29	Fix	0.97	0.98	0.97	594
30	Continuous Jitter	0.09	0.74	0.15	594

Table A.2.1: Precision, recall, f1-score and number of samples per class for the attention base model on the large data set generated using the new specification

Index	Class	Precision	Recall	F1-score	support
31	Continuous Jitter	0.00	0.00	0.00	594
32	Sine	0.63	0.96	0.76	594
33	Sine	0.00	0.00	0.00	594
34	Stagger	0.49	0.95	0.64	594
35	Fix	0.99	1.00	0.99	594
36	Fix	0.99	0.77	0.87	594
37	Stagger	0.00	0.00	0.00	594
38	Stagger	0.98	0.99	0.99	594
39	Stagger	0.00	0.00	0.00	594
40	Fix	0.99	0.97	0.98	594
41	Fix	0.98	0.96	0.97	594
42	Fix	0.96	0.99	0.98	594
43	Fix	0.89	1.00	0.94	594
44	Fix	0.00	0.00	0.00	594
45	Continuous Jitter	0.00	0.00	0.00	594
46	Fix	0.99	0.99	0.99	594
47	Stagger	0.00	0.00	0.00	594
48	Continuous Jitter	0.00	0.00	0.00	594
49	Stagger	0.44	0.99	0.61	594
50	Stagger	0.00	0.00	0.00	594
51	Stagger	0.49	0.97	0.65	594
52	Dwell n' Switch	0.48	0.87	0.62	594
53	Discrete Jitter	0.25	0.99	0.40	594
54	Discrete Jitter	0.00	0.00	0.00	594
55	Stagger	0.98	0.99	0.99	594
56	Stagger	0.98	0.99	0.99	594
57	Fix	0.90	1.00	0.95	594
58	Stagger	0.98	0.99	0.98	594
59	Stagger	0.82	0.97	0.89	594
Accuracy				0.68	35046
Macro avg		0.60	0.68	0.63	35046
Weighted avg		0.60	0.68	0.63	35046

Table A.2.2: Continuation of table A.2.1. Precision, recall, f1-score and number of samples per class for the attention based model on the large data set using the new specification

Index	Class	Precision	Recall	F1-score	Support
1	Dwell n' Switch	0.97	0.97	0.97	594
2	Fix	0.91	0.97	0.94	594
3	Fix	0.89	1.00	0.94	594
4	Discrete Jitter	0.25	0.65	0.36	594
5	Discrete Jitter	0.42	0.94	0.58	594
6	Continuous Jitter	0.00	0.00	0.00	594
7	Continuous Jitter	0.00	0.00	0.00	594
8	Sine	0.00	0.00	0.00	594
9	Sine	0.79	0.99	0.88	594
10	Stagger	1.00	0.01	0.01	594
11	Stagger	0.57	0.97	0.72	594
Accuracy				0.59	6534
Macro avg		0.53	0.59	0.49	6534
Weighted avg		0.53	0.59	0.49	6534

Table A.2.3: The precision, recall, f1-score and number of samples per class for the attention based model on the difficult dataset generated using the new specification

Index	Class	Precision	Recall	F1-score	support
1	Fix	0.19	0.29	0.23	14
2	Fix	0.62	0.35	0.44	23
3	Fix	0.38	0.40	0.39	20
4	Fix	0.50	0.67	0.57	12
5	Discrete Jitter	0.38	0.33	0.35	18
6	Fix	0.70	0.64	0.67	22
7	Fix	0.47	0.44	0.45	16
8	Fix	0.50	0.40	0.44	20
9	Fix	0.14	0.15	0.15	13
10	Stagger	0.42	0.56	0.48	18
11	Continuous Jitter	0.25	0.16	0.19	19
12	Continuous Jitter	0.50	0.44	0.47	16
13	Fix	0.70	0.47	0.56	15
14	Fix	0.25	0.29	0.27	14
15	Fix	0.56	0.56	0.56	16
16	Continuous Jitter	0.60	0.64	0.62	14
17	Continuous Jitter	0.53	0.50	0.52	16
18	Continuous Jitter	0.38	0.43	0.40	14
19	Continuous Jitter	0.43	0.47	0.45	19
20	Continuous Jitter	0.42	0.50	0.46	16
21	Fix	0.36	0.60	0.45	15
22	Continuous Jitter	0.33	0.56	0.42	9
23	Continuous Jitter	0.31	0.33	0.32	12
24	Stagger	0.36	0.31	0.33	16
25	Stagger	0.30	0.27	0.29	11
26	Stagger	0.15	0.17	0.16	18
27	Discrete Jitter	0.25	0.24	0.24	17
28	Dwell n' Switch	0.68	0.54	0.60	24
29	Fix	0.64	0.41	0.50	22
30	Continuous Jitter	0.21	0.38	0.27	13

Table A.2.4: Precision, recall, f1-score and number of samples per class for the KNN on the large data set generated using the new specification

Index	Class	Precision	Recall	F1-score	support
31	Continuous Jitter	0.26	0.33	0.29	15
32	Sine	0.45	0.45	0.45	22
33	Sine	0.50	0.71	0.59	14
34	Stagger	0.42	0.31	0.36	16
35	Fix	0.62	0.80	0.70	10
36	Fix	0.67	0.56	0.61	18
37	Stagger	0.29	0.27	0.28	15
38	Stagger	0.50	0.32	0.39	22
39	Stagger	0.33	0.27	0.30	22
40	Fix	0.75	0.58	0.65	31
41	Fix	0.75	0.50	0.60	18
42	Fix	0.50	0.59	0.54	17
43	Fix	0.59	0.59	0.59	27
44	Fix	0.38	0.35	0.36	17
45	Continuous Jitter	0.36	0.31	0.33	16
46	Fix	0.47	0.39	0.42	18
47	Stagger	0.32	0.40	0.35	15
48	Continuous Jitter	0.65	0.61	0.63	18
49	Stagger	0.28	0.42	0.33	12
50	Stagger	0.82	0.82	0.82	17
51	Stagger	0.38	0.42	0.40	19
52	Dwell n' Switch	0.67	0.50	0.57	16
53	Discrete Jitter	0.43	0.15	0.22	20
54	Discrete Jitter	0.18	0.21	0.19	14
55	Stagger	0.39	0.44	0.41	16
56	Stagger	0.38	0.53	0.44	17
57	Fix	0.56	0.64	0.60	14
58	Stagger	0.38	0.29	0.32	21
59	Stagger	0.30	0.64	0.41	11
Accuracy				0.44	1000
Macro avg		0.44	0.44	0.43	1000
Weighted avg		0.44	0.44	0.44	1000

Table A.2.5: Continuation of table A.2.4. Precision, recall, f1-score and number of samples per class for the KNN on the large dataset generated using the new specification

Index	Class	Precision	Recall	F1-score	Support
1	Dwell n' Switch	0.38	0.38	0.38	109
2	Fix	0.33	0.40	0.36	126
3	Fix	0.42	0.41	0.41	119
4	Discrete Jitter	0.26	0.25	0.25	118
5	Discrete Jitter	0.25	0.20	0.22	123
6	Continuous Jitter	0.26	0.26	0.26	110
7	Continuous Jitter	0.25	0.26	0.25	112
8	Sine	0.39	0.43	0.41	117
9	Sine	0.39	0.37	0.38	133
10	Stagger	0.50	0.51	0.50	111
11	Stagger	0.51	0.49	0.50	127
Accuracy				0.59	6534
Macro avg		0.53	0.59	0.49	6534
Weighted avg		0.53	0.59	0.49	6534

Table A.2.6: he precision, recall, f1-score and number of samples per class for the KNN on the difficult dataset using generated the new specification

TRITA-EECS-EX-2022:560